



天数智芯
Iluvatar CoreX

天数智芯

ixFormer(vLLM) 大模型推理框架使用指南

版本：V4.3.0 (x86)|V4.3.6 (ARM)

日期：2025.7.21

适用产品：天垓 150| 智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 ixFormer(vLLM) 大模型推理框架使用指南	6
1.1 修订记录	6
1.2 ixFormer 推理框架功能说明	7
1.3 安装 ixFormer 推理框架	8
1.4 ixFormer 推理指南	8
1.4.1 如何优化推理	8
1.4.1.1 使用更加高效的基础算子	9
1.4.1.2 使用融合算子	9
1.4.1.3 使用更低精度进行推理	10
1.4.2 内置模型列表	10
1.5 ixFormer 支持的推理框架说明	10
1.5.1 vLLM	11
1.5.1.1 安装天数智芯适配版 vLLM	11
1.5.1.2 使用 vLLM 进行快速推理	11
1.5.1.3 使用 vLLM 进行 Server 形式推理	12
1.5.1.4 开启 vLLM CUDA Graph	14
1.5.1.5 vLLM 支持的模型	14
1.5.2 TGI	14
1.5.2.1 安装天数智芯适配版 TGI	14
1.5.2.2 使用 TGI 进行快速推理	14
1.5.2.3 TGI 支持的模型	16
1.5.3 Xinference	16
1.5.4 Ollama	16
1.6 ixFormer 训练指南	16
1.6.1 调用 ixFormer 算子	16
1.6.2 使用 ixFormer 构建模型	16
1.7 附录 1: ixFormer 支持的模型列表	17
1.7.1 ixFormer 内置模型列表	17
1.7.2 vLLM 支持的模型列表	17
1.7.3 TGI 支持的模型列表	18
1.8 附录 2: ixFormer 推理 Python API	19
1.8.1 ixformer.functions (基础算子)	19
1.8.1.1 bnb_dequant	19
1.8.1.2 bnb_double_quant	20
1.8.1.3 bnb_mm_dequant	20
1.8.1.4 bnb_qgemm	21
1.8.1.5 bnb_quant	22
1.8.1.6 bnb_rowcol_absmax	22
1.8.1.7 conv2d	23
1.8.1.8 gemv	24
1.8.1.9 group_norm	25
1.8.1.10 layer_norm	25
1.8.1.11 solve	27

1.8.1.12	linear	27
1.8.1.13	matmul	28
1.8.1.14	mm	29
1.8.1.15	addmm	29
1.8.1.16	quantized_linear	30
1.8.1.17	quantized_weight_dequant	31
1.8.1.18	weight_quantize	32
1.8.1.19	layer_norm_smoothquant	33
1.8.1.20	residual_layer_norm_smoothquant	34
1.8.1.21	softmax	35
1.8.1.22	reshape_and_cache	35
1.8.1.23	reshape_and_cache_v5	35
1.8.1.24	vllm_llama_mlp	36
1.8.2	ixformer.functions (融合算子)	36
1.8.2.1	silu_and_mul	36
1.8.2.2	gelu_and_mul	37
1.8.2.3	gelu_tanh_and_mul	37
1.8.2.4	rms_norm	38
1.8.2.5	act_bias_mm	39
1.8.2.6	fused_group_norm_silu	39
1.8.2.7	dynamic_scaled_quant_smoothquant	40
1.8.2.8	residual_layer_norm	41
1.8.2.9	fused_addmm_bias_col_act	42
1.8.2.10	residual_rms_norm	43
1.8.2.11	layer_norm_2sb_fused	44
1.8.2.12	scaled_dot_product_attention	45
1.8.2.13	silu_and_mul_smoothquant	46
1.8.2.14	rms_norm_smoothquant	47
1.8.2.15	residual_rms_norm_smoothquant	48
1.8.2.16	layer_norm_2sb_smoothquant	50
1.8.2.17	residual_layer_norm_2sb_smoothquant	51
1.8.2.18	lightllm_apply_penalty	53
1.8.2.19	lightllm_destindex_copy_kv	54
1.8.2.20	lightllm_glm2_rope	54
1.8.2.21	lightllm_tokenattention	55
1.8.2.22	vllm_rotary_embedding	57
1.8.2.23	vllm_moe_topk_softmax	58
1.8.2.24	vllm_cache_ops_reshape_and_cache	59
1.8.2.25	vllm_reshape_and_cache	60
1.8.2.26	vllm_copy_blocks	60
1.8.2.27	vllm_swap_blocks	61
1.8.2.28	vllm_batched_rotary_embedding	61
1.8.2.29	reshape_and_cache_flash	63
1.8.2.30	vllm_moe_align_block_size	64
1.8.2.31	vllm_invoke_fused_moe_kernel	65
1.8.2.32	t5_split_qkv	65
1.8.2.33	t5_split_qkv_update_kv_cache	66

1.8.2.34	tgi_apply_rotary	67
1.8.2.35	tgi_gather_prefill_logprobs	68
1.8.2.36	vllm_reshape_and_cache_v4	69
1.8.2.37	reshape_and_cache_v3	69
1.8.2.38	vllm_gptq_shuffle	70
1.8.2.39	gptq_gemm	71
1.8.2.40	flash_attn_varlen_func	71
1.8.2.41	flash_attn_func	75
1.8.3	ixformer.functions (低精度算子)	77
1.8.3.1	i8w8o32	77
1.8.3.2	w8a16	77
1.8.3.3	wu8a16	78
1.8.3.4	w8a8	80
1.8.3.5	wi4a16	81
1.9	附录 3: ixFormer 训练 Python API	82
1.9.1	ixformer.train.functions (基础算子)	82
1.9.2	ixformer.train.functions (融合算子)	83
1.9.2.1	fused_apply_rotary_pos_emb	83
1.9.2.2	geglu	84
1.9.2.3	residual_bias	84
1.9.2.4	residual_bias_ln	85
1.9.2.5	swiglu	86
1.9.2.6	vocab_parallel_cross_entropy	87
1.9.2.7	fused_rms_norm_affine	90

1 ixFormer(vLLM) 大模型推理框架使用指南

1.1 修订记录

- COREX01-BI430-UG12-01: 2025/7/21
 - vLLM 推理新增开启 vLLM CUDA Graph 功能支持
- COREX01-BI430-UG12-00: 2025/6/30
 - 更新适配的 vLLM 推理框架版本为 v0.8.3
 - 更新使用 vLLM 进行快速推理 中步骤 2 的代码
 - 更新vLLM 支持的模型列表
- COREX01-BI420-UG12-00: 2025/1/24
 - ixFormer 推理框架功能说明:
 - * 更新 ixFormer 推理框架版本为 v0.6.0
 - * 新增 v0.6.0 版本支持特性
 - * 更新vLLM 支持的模型列表
 - 更新ixFormer 推理指南
 - * 删除“ 开启推理模式”、“ 自动替换基础算子”
 - * 更新 ixFormer 内置模型列表
 - 更新适配的 vLLM 推理框架版本为 v0.6.3
 - 更新适配的 TGI 推理框架版本为 v2.1.1
 - 新增安装天数智芯适配版 vLLM 和安装天数智芯适配版 TGI
 - 更新vLLM 支持的模型 和TGI 支持的模型
 - 更新ixFormer 训练指南
 - 更新附录 1: ixFormer 支持的模型列表
 - 更新附录 2: ixFormer 推理 Python API 并新增ixformer.functions (低精度算子)
 - 新增附录 3: ixFormer 训练 Python API
- COREX01-BI411-UG12-01: 2024/11/18
 - 补充ixFormer 推理框架功能说明 , 强调了对 X inference 框架的支持
 - 调整ixFormer 推理框架功能说明 与ixFormer 训练指南 章节的位置
- COREX01-BI411-UG12-00: 2024/8/26
 - 更新安装 ixFormer 推理框架, 提示用户可在 天数智芯官网 下载 ixFormer 的 whl 安装包
- COREX01-BI410-UG12-01: 2024/8/9
 - 新增X inference 和Ollama, 提供限制说明
- COREX01-BI410-UG12-00: 2024/6/30
 - ixFormer 推理框架功能说明:
 - * 更新 ixFormer 推理框架版本为 v0.5.0
 - * 提示用户不建议自行适配天数智芯暂不支持的推理框架

- * 新增 v0.5.0 版本支持特性
 - 更新推理功能**内置模型列表**
 - 新增**ixFormer 训练指南**
 - 更新适配的 vLLM 推理框架版本为 v0.3.3
 - 更新**vLLM 支持的模型列表**
 - 更新适配的 TGI 推理框架版本为 v1.4.3
 - 更新**附录 1: ixFormer 支持的模型列表**
- COREX01-BI340-UG12-00: 2024/3/25

文档本次发布内容与 V3.3.0-BI150 文档相比 (COREX01-BI330-UG12-01), 有以下更新:

- 更新 ixFormer 推理框架的版本为 v0.4.0
- 更新天数智芯适配版 vLLM 的版本为 v0.2.6
- 更新天数智芯适配版 TGI 的版本为 v1.1.0
- 更新**vLLM 推理框架的操作说明**:
 - * 删除原“在 vLLM 中添加新模型”小节, 新增**使用 vLLM 进行 Server 形式推理**小节
 - * 更新**vLLM 支持的模型列表**
- 更新**附录 1: ixFormer 支持的模型列表**

1.2 ixFormer 推理框架功能说明

ixFormer 是天数智芯设计的专用于大模型推理和训练优化的加速框架, 支持业界目前主流大模型的推理加速, **例如 vLLM、TGI、Xinference 等框架**, 实现大模型在天数智芯加速卡上的最佳推理性能。ixFormer 底层使用 C++ 实现, 封装了多个高效算子; 在 Python 层面, ixFormer 提供了 Operation, Layer, Model 等多个层次的 API, 支持 FP16, INT8, INT4 等多种数据结构下的加速。当前提供的 ixFormer 版本是 v0.6.0。

ixFormer 推理框架主要包含以下内容:

- 优化算子库: 基于天数智芯加速卡硬件特性, ixFormer 重新实现了丰富的算子库, 算子库中包括了基础算子、融合算子以及低精度算子用于模型加速。
- 兼容设计: ixFormer 设计兼容 PyTorch 框架, 可以直接使用 ixFormer 算子替换 PyTorch 模型中的性能瓶颈算子, 从而可以在推理中使用并加速 PyTorch 模型。
- 内置推理模型: 借助于高效算子库, ixFormer 提供了高度优化的内置模型, 结合张量模型并行和流水线模型并行, 实现模型性能加速。详细的模型支持列表, 请参考**附录 1: ixFormer 支持的模型列表**。
- 推理框架支持: 得益于框架的兼容性设计, ixFormer 可以使用加速算子对第三方框架进行加速, ixFormer 目前提供了对 vLLM 和 Text Generation Inference (TGI) 框架的推理加速支持。更多详细内容, 请参考**ixFormer 支持的推理框架说明**。

ixFormer 实现了丰富的算子库等构建模型需要的特性, 同时, ixFormer 被设计为兼容 PyTorch。因此, 在推理和训练场景中, 您可以使用 ixFormer 对基于 PyTorch 的推理框架和训练框架进行加速, 从而支持以下重要特性。

- ixFormer v0.6.0 版本特性
 - 重构以规范化代码
 - 添加用于 vLLM 的 flash-attention 接口, wui4a16 量化推理算子支持
 - 添加用于 flashinfer, vLLM 和 TGI 的 paged attention 推理算子支持
 - 删除基础算子自动替换功能

- ixFormer v0.5.0 版本特性
 - 添加 gptq, w8a16, wi4a16 量化推理算子支持
 - 添加混合专家模型推理算子支持
 - 添加 LLaMA 系列模型张量并行 (Tensor Parallelism) 通信计算 Overlap 支持
 - 添加 DeepCache, ComfyUI 框架支持
- ixFormer v0.3.0 版本特性
 - 支持 ixFormer 基础算子加速
 - 支持 ixFormer 接口基础算子自动替换
 - 支持主流大模型推理加速

详细信息，请参考 [ixFormer 推理指南](#)。

1.3 安装 ixFormer 推理框架

本小节旨在介绍 ixFormer 推理框架的安装过程，当前仅支持通过 Python 库安装 ixFormer。

1. 确保您已经成功安装天数智算软件栈，详情请参考《软件栈安装指南》。
2. 获取 ixFormer 的 .whl 包 (在 [天数智芯官网](#) 下载或联系您的应用工程师获取)，获取到的 .whl 包必须和您当前环境的 Python 版本保持一致。

若您使用 Docker 安装软件栈，则无需额外安装 .whl 包，天数智芯提供的 Docker 镜像安装包内已包含 ixFormer 推理框架。

3. 使用 **pip3 install** 命令安装 ixFormer，例如：

```
$ pip3 install ixformer-{ixformer_version}+corex.{v.r.m}-cp310-cp310-linux_{ARCH}.whl
```

4. 安装完成后，在终端运行如下命令检查安装是否成功：

```
$ python3 -c "import ixformer; print(ixformer.__file__)"
```

如果回显版本号，表示安装成功。

1.4 ixFormer 推理指南

为了最大化模型的推理性能，ixFormer 在模型层面设计了高效的基础算子、融合算子和低精度算子，从框架层面设计了高效的推理模式。

本小节旨在指导使用 ixFormer 对模型进行推理，并介绍 ixFormer 内置的推理模型。

1.4.1 如何优化推理

ixFormer 为您提供以下方式对模型推理进行性能优化：

- 使用更加高效的基础算子
- 使用融合算子
- 使用更低精度进行推理

1.4.1.1 使用更加高效的基础算子

对于基础算子，ixFormer 在使用上保持了和 PyTorch 相同的接口。在大部分场景下，您只需要将 `torch.xxx_func` 替换为 `ixformer.functions.xxx_func` 即可使用更高效的基础算子：

```
import numpy as np
import torch
import ixformer

x = torch.randn(2, 64, 32, dtype=torch.half, device="cuda", requires_grad=True)
w = torch.randn(32, 64, dtype=torch.half, device="cuda", requires_grad=True)
# In PyTorch
out = torch.matmul(x, w)
```

上述代码中的 `matmul` 函数可以被替换为 ixFormer 中的函数：

```
# Replace by ixFormer
out = ixformer.functions.matmul(x, w)
```

1.4.1.2 使用融合算子

在多数情况下，ixFormer 实现的融合算子并不存在于 PyTorch 中，因此无法如使用更加高效的基础算子中描述的那样使用 ixFormer 高效的基础算子来替换 PyTorch 的原有算子。此时，您需要手动将 ixFormer 实现的算子在模型中进行替换。

下面以 RMS_norm 的 PyTorch 实例为例进行说明：

```
class RMSNorm(nn.Module):
    def __init__(self, num_features, eps=1e-8):
        super(RMSNorm, self).__init__()

        self.eps = eps
        self.weight = nn.Parameter(torch.ones(hidden_size))

    def forward(self, x):
        mean_square = torch.mean(x**2, dim=-1, keepdim=True)
        norm = torch.sqrt(mean_square + self.eps)
        x = x / norm * self.weight

        return x

# in other model
def __init__(self, num_features):
    self.rms_norm = RMSNorm(num_features)
    ...
def forward(self, x):
    norm_x = self.rms_norm(x)
    ...
```

您可以使用 ixFormer 提供的融合算子进行替换：

```
from ixformer.functions import rms_norm
# in other model
def __init__(self):
    self.norm_weight = ...
    ...
def forward(self, x):
    norm_x = rms_norm(x, self.norm_weight)
    ...
```

目前，ixFormer 提供的融合算子还在持续增加，参考[附录 2：ixFormer 推理 Python API](#)了解详情。

关于融合算子的使用，请参考[ixFormer 支持的推理框架说明](#)中vLLM 相关内容。**需要注意的是**，部分融合算子只用于推理，其反向传播的部分尚未实现。如果您在使用融合算子进行训练的过程中出现问题，可以尝试回退到 PyTorch 版本，或联系您的应用工程师获取解决方案。

1.4.1.3 使用更低精度进行推理

目前，ixFormer 主要支持 FP16 精度进行推理，INT8 和 INT4 精度的推理也支持。得益于 ixFormer 和 PyTorch 的兼容性，您在使用 FP16 进行推理时，无需修改相关代码即可快速推理。对于 INT8 和 INT4 推理，则需要先完成对应的模型量化操作。

1.4.2 内置模型列表

基于 ixFormer 提供的算子库及相关特性，天数智芯提供了如下模型供您进行推理 (向您的应用工程师获取相应代码和使用说明)：

模型	精度
CLIP	FP16
Codeshell-7B-Chat	FP16

目前，ixFormer 内置的模型都被统一存放在 ixFormer 的 Model Zoo 中，您可以在[附录 1：ixFormer 支持的模型列表](#)中查看所有支持的模型 (包括支持的其它推理模型)。

1.5 ixFormer 支持的推理框架说明

得益于框架的兼容性设计，ixFormer 支持使用加速算子对第三方框架进行加速。目前，ixFormer 提供了对 vLLM 和 TGI 框架进行推理加速的支持。

1.5.1 vLLM

通过在天数智芯加速卡上实现 PagedAttention 的张量缓存管理核心机制，ixFormer 提供了 vLLM 框架的推理支持。当前支持的版本为 vLLM v0.8.3。在 vLLM 的模型实现中，大部分算子使用在天数智芯加速卡上实现的高效算子，结合 vLLM 的前端处理，提升模型吞吐量。

1.5.1.1 安装天数智芯适配版 vLLM

- 如果您已在宿主机上安装天数智算软件栈，请在 [天数智芯官网](#) 的 **客户支持 > 资源中心** 页面下载 vLLM 的 whl 包并进行安装，如 `vllm-{version}+corex.{v.r.m}-py3-none-any.whl`
- 如果您使用天数智芯提供的 Docker 镜像生成天数智算软件栈，则无需额外安装 vLLM

1.5.1.2 使用 vLLM 进行快速推理

ixFormer 支持您使用 vLLM 对 **vLLM 支持的模型列表** 中提供的模型进行快速推理。

示例

下面以 LLaMA2-13B 模型为例，说明如何使用 vLLM 进行快速推理。

从 [Hugging Face](#) 或天数智芯提供的文件 (向您的应用工程师获取相关文件) 中获取模型权重，并将其存放至环境的 `/home/data/llama2/llama2-13b` 目录下。ixFormer 支持的 vLLM 推理框架保持了同开源框架一致的使用流程。

1. 首先，从 ixFormer 中导入必要的模块：

```
from vllm import LLM, SamplingParams
```

2. 利用 LLM 和 SamplingParams 分别创建执行推理的模型实例和生成文本管理的参数实例：

```
llm = LLM(model="/home/data/llama2/llama2-13b",
          gpu_memory_utilization=0.9,
          max_model_len=2048,
          tokenizer_mode="slow",
          tensor_parallel_size=1
        )
sampling_params = SamplingParams(temperature=0.8, top_p=0.95,
                                max_tokens=1024,
                                )
```

在上述代码中，

- LLM 的初始化函数会根据模型权重位置自动加载模型配置文件
- `gpu_memory_utilization` 用于控制加速卡内存的占用比例
- `max_num_batched_tokens` 用于控制模型最大的输入 tokens 数量
- `tensor_parallel_size` 用于控制模型张量并行度

如果使用两张加速卡进行模型的并行推理，则可以设置 `tensor_parallel_size=2`。更多详细的参数信息，请参考源代码文件。

3. 设置输入文本并进行推理：

```
prompts = [
    "Shanghai is one of the most prosperous cities in China,"
]
outputs = llm.generate(prompts, sampling_params)
prompt = outputs[0].prompt
generated_text = outputs[0].outputs[0].text
```

generate 函数完成输入文本到 tokens 的转换、执行推理以及输出 tokens 到输出文本的转换，通过打印模型的输入输出，获得结果：

```
print(generated_text)
>>>
and it is also the largest city in the country. While visiting Shanghai, it is
↳ important to be aware of the local laws and regulations, including those related
↳ to traffic safety. In Shanghai, traffic safety is a top priority, and the local
↳ authorities have implemented various measures to ensure the safety of ped
```

总结

上述流程是使用 vLLM 进行离线推理的主要流程，更多详细内容，请参考 [vLLM 官方网站](#) 或 [附录 1：ixFormer 支持的模型列表](#) (向您的应用工程师获取模型推理相应代码和使用说明)。

1.5.1.3 使用 vLLM 进行 Server 形式推理

vLLM 提供了两种 Server 推理方式，第一种为 vLLM 自身实现的 Server，可以用于 vLLM 模型的快速使用和验证，但功能性略不足。第二种为官方建议的使用基于 OpenAI 接口的 Server 推理。下面将分别介绍两种推理方式。

方式一：vLLM 自身实现的 Server 推理

1. 首先，通过以下命令启动 Server：

```
python3 -m vllm.entrypoints.api_server --model /path/to/node --host 127.0.0.1 --port
↳ 12345
```

其中，主要设置模型文件路径以及 Server 的启动地址，其他参数与快速推理中的相同。

2. 使用 curl 进行快速验证：

```
curl -s 127.0.0.1:12345/generate \
-X POST \
-d '{
    "prompt": "Shanghai is one of the most prosperous cities in China,",
    "temperature": 0.0,
    "max_tokens": 128,
    "stream": "true"}' \
-H 'Content-Type: application/json'
```

或者，您也可以通过 Python 脚本进行使用：

```
import requests

headers = {"User-Agent": "Test Client"}
pload = {
    "prompt": "Shanghai is one of the most prosperous cities in China,",
    "temperature": 0.0,
    "max_tokens": 128,
    "stream": False,
}
response = requests.post(http://127.0.0.1:12345/generate, headers=headers, json=pload)
data = json.loads(response.content)
output = data["text"]

print(output)
```

方式二：使用基于 OpenAI 接口的 Server 推理

由于 vLLM 自身的 Server 推理框架提供的功能较少，因此，建议您使用 OpenAI 进行 Server 推理。

1. 首先，通过以下命令启动 Server：

```
python3 -m vllm.entrypoints.openai.api_server --model /path/to/node --host 127.0.0.1
↪ --port 12345
```

2. 使用 **curl** 进行快速验证：

```
curl http://127.0.0.1:12345/v1/models

curl -s 127.0.0.1:12345/v1/completions \
-H "Content-Type: application/json" \
-d '{"model": "/model/name/",
    "prompt": "Shanghai is one of the most prosperous cities in China,",
    "temperature": 0.0,
    "max_tokens": 128,
    "stream": "true"}'
```

其中，/model/name/ 为通过第一行命令查询得到的正在运行的模型 ID，之后便可以发起请求。

或者，您也可以通过 Python 脚本进行使用：

```
from openai import OpenAI

openai_api_key = "EMPTY"
openai_api_base = "http://127.0.0.1:12345/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)

models = client.models.list()
```

```
model = models.data[0].id

completion = client.completions.create(
    model=model,
    prompt="Shanghai is one of the most prosperous cities in China,",
    echo=False,
    stream=False)

print(completion)
```

OpenAI 同时提供了 AsyncClient, Chat 接口等功能, 详情请参考 [OpenAI 官方文档](#)。

1.5.1.4 开启 vLLM CUDA Graph

使用 CUDA Graph 功能可进一步提升推理吞吐量。您可以在执行 vLLM 推理前, 导入环境变量开启 CUDA Graph:

```
$ export VLLM_ENFORCE_CUDA_GRAPH=1
```

1.5.1.5 vLLM 支持的模型

参考[vLLM 支持的模型列表](#)。

1.5.2 TGI

TGI (Text Generation Inference) 是 HuggingFace 支持的推理部署工具, 具有和 vLLM 类似的功能。ixFormer 同样提供了对 TGI 框架的推理支持, 并支持使用 ixFormer 算子对模型进行加速。当前支持的版本为 TGI v2.1.1。

1.5.2.1 安装天数智芯适配版 TGI

- 如果您已在宿主机上安装天数智算软件栈, 请在 [天数智芯官网](#) 的 [客户支持](#) > [资源中心](#) 页面下载 TGI 的 run 包并进行安装, 如 text-generation-inference-{version}.run
- 如果您使用天数智芯提供的 Docker 镜像生成天数智算软件栈, 则无需额外安装 TGI

1.5.2.2 使用 TGI 进行快速推理

ixFormer 支持您使用 vLLM 对[TGI 支持的模型列表](#)中提供的模型进行快速推理。

示例

下面以 LLaMA-13B 模型为例, 说明如何使用 TGI 进行快速推理。

从 [Hugging Face](#) 或天数智芯提供的文件 (向您的应用工程师获取相关文件) 中获取模型权重, 并将其存放至环境的 /home/data/llama/llama-13b 目录下。

1. 在运行服务前，需要先安装 Protoc 和 Rust 用于前端服务：

- 安装 Protoc：

```
# PROTOC_ZIP=protoc-21.12-linux-{ARCH}.zip
$ curl -OL https://github.com/protocolbuffers/protobuf/releases/download/
  ↪ v21.12/$PROTOC_ZIP
$ sudo unzip -o $PROTOC_ZIP -d /usr/local bin/protoc
$ sudo unzip -o $PROTOC_ZIP -d /usr/local 'include*\'
```

- 安装 Rust：

```
$ curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

2. 在本例中，使用 Server 方式进行模型推理。

- 对于单卡：

```
$ export PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python
$ CUDA_VISIBLE_DEVICES=0 USE_FLASH_ATTENTION=true \
  text-generation-launcher --model-id /home/data/llama/llama-13b/ \
  --sharded false --dtype float16 --disable-custom-kernels \
  --trust-remote-code --max-batch-total-tokens 4096 --port 4000
```

- 对于多卡：

```
$ export PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python
$ CUDA_VISIBLE_DEVICES=0,2 USE_FLASH_ATTENTION=true text-generation-launcher
  ↪ --model-id /home/data/llama/llama-13b/ \
  --sharded true --num-shard 2 --dtype float16 --disable-custom-kernels \
  --trust-remote-code --max-batch-total-tokens 4096 --port 4000
```

与在 vLLM 中演示的离线推理不同，以 Server 方式运行推理的方式不需要显式编写 Python 文件的过程，vLLM 同样支持以 Server 方式进行推理。在 Server 方式下，您可以直接使用 curl 等发起服务请求：

- 非流式服务请求：

```
$ curl 127.0.0.1:4000/generate \
  -X POST \
  -d '{"inputs":"Q: What is the largest animal?
  ↪ \nA:", "parameters":{"max_new_tokens":64}}' \
  -H 'Content-Type: application/json'
```

- 流式服务请求：

```
$ curl 127.0.0.1:4000/generate_stream \
  -X POST \
  -d '{"inputs":"Q: What is the largest animal?
  ↪ \nA:", "parameters":{"max_new_tokens":20}}' \
  -H 'Content-Type: application/json'
```

总结

以上是基于 ixFormer 进行 TGI Server 推理的主要流程，详细的使用流程请参考 [TGI 官方网站](#)。

1.5.2.3 TGI 支持的模型

参考TGI 支持的模型列表。

1.5.3 Xinference

目前天数智芯只支持 Transformers 和 vLLM 后端，使用下面命令安装 Xinference：

```
$ pip install --upgrade-strategy only-if-needed "xinference[transformers,vllm]"
```

1.5.4 Ollama

Ollama 后端仅支持 llama.cpp，目前天数智算软件栈未适配 llama.cpp，因此不支持 Ollama。

1.6 ixFormer 训练指南

ixFormer 设计兼容 PyTorch 框架并且实现了基础算子库、融合算子库的前向传播和反向传播，可以实现在训练过程中调用 ixFormer 算子以及搭建模型等内容。

1.6.1 调用 ixFormer 算子

ixFormer 提供了构建模型所需的基础算子，如 matmul, swiglu, normalization 等，您可以使用 ixFormer 提供的算子进行计算。调用算子方法如下：

```
import torch
import ixformer.train.functions as ixff

x = torch.randn(2, 64, 32, device="cuda", dtype=torch.half)
weight = torch.randn(32, 64, device="cuda", dtype=torch.half)
out = ixff.matmul(x, weight)
```

1.6.2 使用 ixFormer 构建模型

如果您熟悉 PyTorch，您也会熟悉下面的方法使用 ixFormer 构建模型：

```
import numpy as np
import ixformer.train.functions as ixff

class Linear:
    def __init__(self, in_channels, out_channels):
        self.weight = torch.randn(out_channels, in_channels, dtype=torch.half).cuda()
```

```
self.bias = torch.zeros(out_channels, dtype=torch.half).cuda()
def forward(self, x):
    return ixff.linear(x, self.weight, self.bias)
```

```
x = torch.randn(2, 64, 32, device="cuda", dtype=torch.half)
fc = Linear(32, 64)
out = fc.forward(x)
```

1.7 附录 1: ixFormer 支持的模型列表

ixFormer 提供的 Model Zoo 主要包括 ixFormer 内置模型、vLLM 模型和 TGI 模型。其中 **Official** 是 ixFormer 的内置模型，同时，**Official** 中的模型保持了和官方一致的接口和功能。

ixFormer 还在持续发展中，将会有更多的模型被列入支持列表中。

Note

- 请向您的应用工程师获取下表中的模型代码和使用说明。
- 列表中的模型为当前通过验证的模型。

1.7.1 ixFormer 内置模型列表

- CLIP
- Codeshell-7B-Chat

1.7.2 vLLM 支持的模型列表

- Aquila
- Aquila2
- Baichuan
- Baichuan2
- BART
- BLOOM
- ChatGLM
- Command-R
- DeciLM
- DeepSeek
- DeepSeek-V2
- DeepSeek-V3
- Dolly V2
- EXAONE-3
- Falcon
- Gemma
- Gemma2
- GPT-2

- GPT-J
- GPT-NeoX
- InternLM
- InternLM2
- InternLM3
- Jais
- LLaMA
- LLaMA2
- LLaMA3
- Yi
- MiniCPM
- MiniCPM3
- Minitron
- Mistral
- Mistral
- Mixtral
- MPT
- Nemotron-3
- Nemotron-4
- OPT
- OPT-IML
- OLMo
- OLMo2
- OLMoE
- Orion
- Phi
- Phi-3
- Phi-3-Small
- Persimmon
- PowerLM
- PowerMoE
- Qwen
- Qwen2
- Qwen2MoE
- Qwen3
- SantaCoder
- StableLM
- StarCoder2
- Solar Pro
- TeleChat2
- WizardCoder
- XVERSE

1.7.3 TGI 支持的模型列表

- BLOOM

- ChatGLM
- CodeGeeX2
- Falcon
- Flan-t5
- Gemma
- GPT BigCode
- GPT-NeoX
- Galactical
- LLaMA-2
- Mixtral
- MPT
- OPT
- Qwen2
- Starcoder2

1.8 附录 2: ixFormer 推理 Python API

ixFormer 推理 Python API 包含:

- `ixformer.functions` (基础算子)
- `ixformer.functions` (融合算子)
- `ixformer.functions` (低精度算子)

1.8.1 ixformer.functions (基础算子)

1.8.1.1 bnb_dequant

```
def bnb_dequant(
    qA: torch.Tensor,
    SA: torch.Tensor,
    training: bool = False,
    scale: float = 127.0,
    dequant_type: int = 0,
) -> torch.Tensor:
```

参数说明

- qA - quant input shape: [row, col], dtype: torch.int8
- SA - scale vector dtype: torch.half
- training - 是否用于训练
- scale - 量化位数中所能表示的最大绝对值
- dequant_type - 如果值为 0, every row shared a scale, SA shape: [row]; 如果值为 1, every col shared a scale, SA shape: [col]

返回值

- Tensor - dequant output shape: [row, col], dtype: torch.half

等价实现

```
def ref_bnb_dequant(
    qA: torch.Tensor,
    SA: torch.Tensor,
    training: bool = False,
    scale: float = 127.0,
    quant_type: int = 0,
):
    A = qA * (SA.to(torch.float) / scale).to(SA.dtype)
    return A
```

1.8.1.2 bnb_double_quant

```
def bnb_double_quant(
    A: torch.Tensor, training: bool = False, threshold: float = 0.0
) -> torch.Tensor:
```

参数说明

- A - input, shape: [row, col], dtype: torch.half
- training - 是否用于训练
- scale - 量化位数中所能表示的最大绝对值
- threshold - float, abs of element exceeds threshold will be ignored

返回值

- out_row - quant output, 以 row 量化, shape: [row, col] shape: torch.int8
- out_col - quant output, 以 col 量化, shape: [row, col] shape: torch.int8
- row_stats - shape: [row], 按 row 计算得到的最大绝对值
- col_stats - shape: [col], 按 col 计算得到的最大绝对值
- coo_tensor

1.8.1.3 bnb_mm_dequant

```
def bnb_mm_dequant(
    A: torch.Tensor,
    quant_state: tuple,
    row_stats: torch.Tensor,
    col_stats: torch.Tensor,
    bias: torch.Tensor = None,
    add_bias: bool = False,
    training: bool = False,
)
```

参数说明

- A - input, shape: [row, col], dtype: torch.half
- quant_state
- row_stats - shape: [row], dtype: torch.float
- col_stats - shape: [col], dtype: torch.float
- bias - shape: [col], dtype: torch.half
- add_bias - bool, 是否应用 bias
- training - bool, 是否用于训练

返回值

- Tensor - output, shape: [row, col] dtype: torch.half

1.8.1.4 bnb_qgemm

```
def bnb_qgemm(
    qA: torch.Tensor,
    qW: torch.Tensor,
    SA: torch.Tensor,
    SW: torch.Tensor,
    training: bool = False,
    scaleA: float = 127.0,
    scaleW: float = 127.0,
) -> torch.Tensor:
```

参数说明

- qA - quant input, shape: [bs, in_feature], dtype: torch.int8
- qW - quant weight, shape: [out_feature, in_feature], dtype: torch.int8
- SA - scale vector of qA, shape: [bs], dtype: torch.half
- SW - scale vector of qW, shape: [out_feature], dtype: torch.half
- scaleA - A 量化位数中所能表示的最大绝对值
- scaleW - W 量化位数中所能表示的最大绝对值

返回值

- output - dequant, output shape: [bs, out_feature], dtype: torch.half

等价实现

```
def ref_bnb_qgemm(
    qA: torch.Tensor,
    qW: torch.Tensor,
    SA: torch.Tensor,
    SW: torch.Tensor,
    training: bool = False,
    scaleA: float = 127.0,
    scaleW: float = 127.0,
```

```

):
    y = torch.nn.functional.linear(qA.to(torch.float), qW.to(torch.float))
    out = y * (SA.to(torch.float) / scaleA) * (SW.to(torch.float) / scaleW)
    return out.to(SA.dtype)

```

1.8.1.5 bnb_quant

```

def bnb_quant(
    A: torch.Tensor,
    SA: torch.Tensor,
    training: bool = False,
    scale: float = 127.0,
    quant_type: int = 0,
) -> torch.Tensor:

```

参数说明

- A - quant input shape: [row, col], dtype: torch.half
- sA - scale vector, dtype: torch.half
- training - 是否用于训练
- scale - 量化位数中所能表示的最大绝对值
- quant_type - 如果值为 0, every row shared a scale, SA shape: [row]; 如果值为 1, every col shared a scale, SA shape: [col]

返回值

- Tensor - quant output shape: [row, col], dtype: torch.int8

等价实现

```

def ref_bnb_quant(
    A: torch.Tensor,
    SA: torch.Tensor,
    training: bool = False,
    scale: float = 127.0,
    quant_type: int = 0,
):
    qA = torch.round(A * (scale / SA.to(torch.float))).to(torch.int8)
    return qA

```

1.8.1.6 bnb_rowcol_absmax

```

def bnb_rowcol_absmax(
    input: torch.Tensor,
    training: bool = False,

```

```
threshold: float = 0.0,
type: int = 0,
) -> torch.Tensor:
```

参数说明

- input - input shape: [row, col], 目前 col 值必须满足 col%2==0, dtype: torch.half
- training - 是否用于训练
- threshold - float, abs of element exceeds threshold will be ignored
- type - int, 0 代表以行求绝对最大值, 1 代表以列求绝对最大值

返回值

- Tensor - absmax output, shape: [row], dtype: torch.half

等价实现

```
def ref_bnb_rowcol_absmax(
    input: torch.Tensor,
    training: bool = False,
    threshold: float = 0.0,
    type: int = 0,
):
    if type == 0:
        out = torch.amax(torch.abs(input), dim=1)
    else:
        out = torch.amax(torch.abs(input), dim=0)
    return out
```

1.8.1.7 conv2d

```
def conv2d(
    input: torch.Tensor,
    weight: torch.Tensor,
    bias: torch.Tensor = None,
    stride: Union[int, tuple] = 1,
    padding: Union[int, tuple] = 0,
    dilation: Union[int, tuple] = 1,
    groups: int = 1,
):
```

参数说明

该模块支持 float16

- input - [n,in_c,h,w], dtype: torch.float16
- weight - [out_c,in_c/groups,kH,kW], dtype: torch.float16
- bias - [out_c], dtype: torch.float16, 默认为 None

- stride - Stride of the convolution. Default: 1
- padding - Padding added to all four sides of the input. Default: 0
- dilation - Spacing between kernel elements. Default: 1
- groups - Number of blocked connections from input channels to output channels. Default: 1

返回值

- $h_{out} = (h_{in} + 2 * pad_h - dilation_h * (kernel_h - 1) - 1) / stride_h + 1$;
- $w_{out} = (w_{in} + 2 * pad_w - dilation_w * (kernel_w - 1) - 1) / stride_w + 1$;
- Tensor - [n,out_c,h_out,w_out], dtype: torch.float16

等价实现

```
import torch.nn.functional as NNF
def ref_conv2d(
    input: torch.Tensor,
    weight: torch.Tensor,
    bias: torch.Tensor = None,
    stride: Union[int, tuple] = 1,
    padding: Union[int, tuple] = 0,
    dilation: Union[int, tuple] = 1,
    groups: int = 1,
):
    output = NNF.conv2d(input, weight, bias, stride, padding, dilation, groups)
    return output
```

1.8.1.8 gemv

```
def gemv(x: torch.Tensor, A: torch.Tensor, gemv_max_batch: int = 1):
```

参数说明

该模块支持 float16, bfloat16

- x - [* , k], input, dtype: [torch.half, torch.bfloat16]
- A - [n, k], weight, dtype: [torch.half, torch.bfloat16]
- gemv_max_batch - int, 用于是否满足 gemv 使用条件的判断

gemv 的使用条件 (input: [m, k], weight: [n, k]):

- m ≤ gemv_max_batch
- k % 2 == 0 n % 2 == 0
- bias is None

返回值

- Tensor - [* , n], dtype: [torch.half, torch.bfloat16]

等价实现

```
def ref_gemv(x: torch.Tensor, A: torch.Tensor, gemv_max_batch: int = 1):
    output = torch.nn.functional.linear(x, A)
    return output
```

1.8.1.9 group_norm

```
def group_norm(
    input: torch.Tensor,
    num_groups: int,
    weight: torch.Tensor,
    bias: torch.Tensor,
    eps: float = 1e-05,
    is_nhw: bool = False, #
    act_type: int = 0,
):
```

参数说明

该模块支持 float16

- input - "contiguous_format": [n,c,h,w] or [n,c,h] "channels_last": [n,h,w,c]
- num_groups - int, number of groups to separate the channels into
- weight - [C], affine parameters
- bias - [C], affine parameters
- eps - a value added to the denominator for numerical stability. Default: 1e-5
- is_nhw - Determine whether the input shape is [n,h,w,c]
- act_type - 0 or 1, if act_type=1, silu

返回值

- Tensor - same shape as input, "contiguous_format": [n,c,h,w] or [n,c,h] "channels_last": [n,h,w,c]

等价实现

```
def ref_group_norm(input, num_groups, weight, bias, eps):
    output = torch.nn.functional.group_norm(input, num_groups, weight, bias, eps)
    return output
```

1.8.1.10 layer_norm

```
def layer_norm(
    input: torch.Tensor,
    normalized_shape: List[int],
    ln_weight: torch.Tensor,
    ln_bias: torch.Tensor,
    eps: float = 1e-5,
)
```

参数说明

该模块支持 float16, bfloat16

- input - [batch_count * seq_len, hidden_size]
- normalized_shape - list[int], [hidden_size]
- ln_weight - [hidden_size], the learnable weights of the module of shape normalized_shape
- ln_bias - [hidden_size], the learnable weights of the module of shape normalized_shape
- eps - a value added to the denominator for numerical stability. Default: 1e-5

返回值

- Tensor - [batch_count * seq_len, hidden_size, hidden_size] (same shape as input)

等价实现

```
def ref_layer_norm(
    input: torch.Tensor,
    normalized_shape: List[int],
    ln_weight: torch.Tensor,
    ln_bias: torch.Tensor,
    eps: float = 1e-5,
):
    if not (
        input.dtype == torch.float16 or input.dtype == torch.bfloat16
    ):
        raise NotImplementedError("layer_norm only support data format of float16 now!")
    # normalized_shape 需要是 list 或者 tuple, 并且不能为空
    norm_size = 1
    if normalized_shape == None:
        norm_size = ln_weight.size(-1)
        normalized_shape = [norm_size]
    else:
        if (
            isinstance(normalized_shape, list) or isinstance(normalized_shape, tuple)
        ) and len(normalized_shape) >= 1:
            for i in normalized_shape:
                norm_size = i * norm_size
        else:
            raise f"layer_norm(): argument 'normalized_shape' (position 2) must be tuple
                of ints, not {type(normalized_shape)}"

    if torch.numel(input) % norm_size != 0:
        raise f"Given normalized_shape={normalized_shape}, expected input with shape [*,"
            f" {norm_size}], but got input of {input.shape}"

    output = torch.nn.functional.layer_norm(
        input, normalized_shape, ln_weight, ln_bias, eps=eps
    )

    return output.to(input.dtype)
```

1.8.1.11 solve

```
def solve(
    A: torch.Tensor, B: torch.Tensor, *, left: bool = True, out: torch.Tensor = None
):
```

参数说明

该模块支持 float32, 目前当 left = False 时, 计算错误, 只支持 left = True

- A - tensor of shape (*, n, n) where * is zero or more batch dimensions.
- B - right-hand side tensor of shape (, n) or (, n, k) or (n,) or (n, k)
- left - whether to solve the system AX=B or XA=B. Default: True.
- out - output tensor. Ignored if None. Default: None.

返回值

- xout - (*, n, k)

等价实现

```
def ref_solve(
    A: torch.Tensor, B: torch.Tensor, *, left: bool = True, out: torch.Tensor = None
):
    out = torch.linalg.solve(A, B, left=left)
    return out
```

1.8.1.12 linear

```
def linear(
    input: torch.Tensor,
    weight: torch.Tensor,
    bias: torch.Tensor = None,
):
```

参数说明

该模块支持 float16, bfloat16

- input -[* ,k] dtype: [torch.half, torch.bfloat16]
- weight - [n,k] dtype: [torch.half, torch.bfloat16]
- bias - [n] dtype:[torch.half, torch.bfloat16]

详情参考 [torch.nn.functional.linear](#)

返回值

- output -[* ,n], dtype: [torch.half, torch.bfloat16]

等价实现

```
def ref_linear(
    input: torch.Tensor,
    weight: torch.Tensor,
    bias: torch.Tensor = None,
):
    output = torch.nn.functional.linear(input, weight, bias)
    return output
```

1.8.1.13 matmul

```
def matmul(
    input: torch.Tensor,
    other: torch.Tensor,
    *,
    transa: bool = False,
    transb: bool = False,
    alpha: float = 1.0,
) -> torch.Tensor
```

参数说明

该模块支持 float16

- input - 矩阵 A [..., m, k] if transa == False, if transa==True, 矩阵 A [..., k, m]
- other - 矩阵 B [..., k, n] if transb == False, if transb == True, 矩阵 B [..., n, k]
- out - 矩阵 C [..., m, n]
- transa - 是否转置 A
- transb - 是否转置 B
- alpha - Gemm 中的 Alpha 系数

等价实现

```
def ref_matmul(input, other, *, transa, transb, alpha):
    if transa:
        dims = list(range(input.ndim))
        dims[-1], dims[-2] = dims[-2], dims[-1]
        input = input.permute(*dims).contiguous()

    if transb:
        dims = list(range(other.ndim))
        dims[-1], dims[-2] = dims[-2], dims[-1]
        other = other.permute(*dims).contiguous()

    return alpha * torch.matmul(input, other)
```

1.8.1.14 mm

```
def mm(
    input:torch.Tensor,
    mat:orch.Tensor,
    *,
    out:torch.Tensor=None)
```

参数说明

该模块支持 float16, bfloat16, float32

- input - (n,m), the first matrix to be matrix multiplied
- mat - (m,p), the second matrix to be matrix multiplied
- out - (n,p)

返回值

- out - (n,p), the output tensor

等价实现

```
def ref_mm(input, mat, *, out=None):
    out = torch.mm(input, mat, out)
    return out
```

1.8.1.15 addmm

```
def addmm(
    input:torch.Tensor,
    mat1:orch.Tensor,
    mat2:orch.Tensor,
    *,
    beta: float =1,
    alpha: float =1,
    out:torch.Tensor=None)
```

参数说明

该模块支持 float16, bfloat16, float32

- input - (n,p), matrix to be added
- mat1 -(n,m), the first matrix to be matrix multiplied
- mat2 - (m,p), the second matrix to be matrix multiplied
- beta - multiplier for input
- alpha - multiplier for `mat1@mat2`
- out - (n,p)

返回值

- out - (n,p), the output tensor

等价实现

```
def ref_addmm(input, mat1, mat2, *, beta=1, alpha=1, out=None):
    output_pt = torch.addmm(input, mat1, mat2, alpha=alpha, beta=beta)
    return output_pt
```

1.8.1.16 quantized_linear

```
def quantized_linear(
    inputs: torch.Tensor,
    qweights: torch.Tensor,
    scales: torch.Tensor,
    quant_type: str,
    bits: int,
    qzeros: torch.Tensor = None,
    bias: torch.Tensor = None,
    group_size: int = -1,
    g_idx: torch.Tensor = None,
    format: str = "unknown",
):
```

参数说明

- QuantType = awq
 - inputs - (bs, ic) bf16/fp16
 - qweights - int32 NN:(ic, oc // 8) TN:(oc, ic // 8)
 - scales - (ic // group_size, oc)fp16/bf16
 - qzeros - int32(ic // group_size, oc // 8)
 - bias - (oc) fp16/bf16
 - group_size - 64/128
 - format - TN/NN
 - ApiCall - vllm & auto-awq
- QuantType = gptq
 - inputs - (bs, ic) bf16/fp16
 - qweights - int32 (ic//8, oc)
 - scales - (ic // group_size, oc)fp16/bf16
 - qzeros - int32(ic // group_size, oc // 8)
 - bias - (oc) fp16/bf16
 - group_size - 64/128
 - format -
 - ApiCall - auto-gptq
- QuantType = fp4

- inputs - (bs, ic) bf16/fp16
- qweights - uint8 (oc * ic // 2, 1)
- scales - (oc * ic // group_size)fp32
- qzeros -
- bias - (oc) fp16/bf16
- group_size - 64/128
- format -
- ApiCall - bitsandbytes
- QuantType = nf4
 - inputs - (bs, ic) bf16/fp16
 - qweights - uint8 (oc * ic // 2, 1)
 - scales - (oc * ic // group_size)fp32
 - qzeros -
 - bias - (oc) fp16/bf16
 - group_size - 64/128
 - format -
 - ApiCall - bitsandbytes
- QuantType = int8
 - inputs - (bs, ic) bf16/fp16
 - qweights - int8 TN:(oc, ic) NN:(ic, oc)
 - scales - (1, oc)fp16/bf16
 - qzeros -
 - bias - (oc) fp16/bf16
 - group_size
 - format - TN
 - ApiCall - vllm & bitsandbytes

返回值

- Tensor - output tensor, shape:(bs,oc), dtype: float16/bfloat16

1.8.1.17 quantized_weight_dequant

```
def quantized_weight_dequant(
    qweights: torch.Tensor,
    scales: torch.Tensor,
    quant_type: str,
    output_type: str,
    bits: int,
    qzeros: torch.Tensor = None,
    group_size: int = -1,
    g_idx: torch.Tensor = None,
)
```

参数说明

- qweights - quant weight, dtype:uint8/int32
- scale - scale vector of qweights, dtype:fp32/fp16/bf16
- quant_type - fp4/nf4/gptq/gptq-ex, dtype:str
- output_type - fp16/bf16, dtype:str
- bits - 如果 quant_type 为 fp4/nf4, 此参数不需要, dtype:int; 如果 quant_type 为 gpt/gptq-ex, bits 支持 4/8
- qzeros - 如果 quant_type 为 fp4/nf4, 此参数不需要, dtype:int32
- group_size - dtype:int
- g_idx - 如果 quant_type 为 fp4/nf4, 此参数不需要, dtype:int32
- QuantType = fp4/nf4
 - qweights - (oc, ic/2) uint8
 - scales - (oc * ic/group_size) fp32
 - qzeros
- QuantType = gptq/gptq-ex
 - qweights - (ic// (32/bits), oc) int32
 - scales - (ic // group_size, oc) fp16/bf16
 - qzeros - (ic // group_size, oc // (32/bits)) int32

返回值

- Tensor - dequant weight output, if quant_tpye= fp4/nf4, shape: (oc,ic), 否则为 (ic,oc), dtype:float16,bfloat16

1.8.1.18 weight_quantize

```
def weight_quantize(
    A: torch.Tensor,
    absmx: torch.Tensor,
    out: torch.Tensor,
    blocksize: int,
    n: int,
    quant_dtype: str,
    code: torch.Tensor = None,
):
```

参数说明

- A - shape: [row, col], dtype: [torch.half, torch.bfloat16, torch.float]
- absmx - shape: [row], dtype: torch.float32
- quant_dtype - str, "int8"/"fp4"/"nf4"
- out - shape: [row, col], dtype: torch.int8
- blocksize - int
- n - int
- code - 只在 torch.bfloat16 的情况下生效

返回值

- Tensor - output Tensor, shape: [row ,col], dtype: torch.int8

1.8.1.19 layer_norm_smoothquant

```
def layer_norm_smoothquant(
    input: "torch.Tensor",
    weight: "torch.Tensor",
    bias: "torch.Tensor",
    smooth_scales: "torch.Tensor",
    i8_output: "torch.Tensor"=None,
    output_scales: "torch.Tensor"=None,
    eps=1e-5
)
```

参数说明

该模块支持 float16, bfloat16

- input (Tensor) - shape (...),K), dtype: float16,bfloat16
- weight (Tensor) - shape (K), dtype: float16,bfloat16
- bias (Tensor) - shape (K), dtype: float16,bfloat16
- smooth_scales (Tensor) - shape (K), smooth scale factors, dtype: float16,bfloat16
- i8_output(optional Tensor) - shape (...),K).
- output_scales(optional Tensor) - shape (...)
- eps - a value added to the denominator for numerical stability. Default: 1e-5

返回值

- i8_output(optional Tensor) - shape (...),K), dtype int8
- output_scales(optional Tensor) - shape (...), dtype float32

等价实现

```
def ref_layer_norm_smoothquant(input, weight, bias, smooth_scales, i8_output=None,
    ↪ output_scales=None, eps=1e-5):
    input = torch.nn.functional.layer_norm(
        input, [weight.shape[-1]], weight, bias, eps=eps
    )

    return ref_dynamic_scaled_quant_smoothquant(input, smooth_scales, i8_output,
    ↪ output_scales)
def ref_dynamic_scaled_quant_smoothquant(input, smooth_scales, i8_output=None,
    ↪ output_scales=None):
    scales_shape = input.shape[:-1]
    input = input.float()
    output = input * smooth_scales.view(1, -1)
    amax_, _ = torch.max(torch.abs(output), dim=-1, keepdim=True)
    scales = amax_ / 127.0
    output = output / scales
    output = torch.clamp(torch.round(output), -127, 127).to(torch.int8)
    if i8_output is not None:
        i8_output.copy_(output)
```

```

        output = i8_output
    if output_scales is not None:
        output_scales.view(-1).copy_(scales.view(-1))
        scales = output_scales

    return output, scales.view(scales_shape)

```

1.8.1.20 residual_layer_norm_smoothquant

```

def residual_layer_norm_smoothquant(
    input: "torch.Tensor",
    residual: "torch.Tensor",
    weight: "torch.Tensor",
    bias: "torch.Tensor",
    smooth_scales: "torch.Tensor",
    i8_output: "torch.Tensor"=None,
    output_scales: "torch.Tensor"=None,
    eps=1e-5
)

```

参数说明

该模块支持 float16, bfloat16

- input (Tensor) - shape (...),K), dtype float16,bfloat16
- residual (Tensor) - shape (...),K), dtype float16,bfloat16
- weight (Tensor) - shape (K), dtype float16,bfloat16
- bias (Tensor) - shape (K), dtype float16,bfloat16
- smooth_scales (Tensor) - shape (K), smooth scale factors, dtype float16,bfloat16
- i8_output (optional Tensor) - shape (...),K).
- output_scales (optional Tensor) - shape (...)
- eps - a value added to the denominator for numerical stability. Default: 1e-5

返回值

- residual (Tensor) - shape (...),K), dtype float16,bfloat16
- i8_output (optional Tensor) - shape (...),K), dtype int8
- output_scales (optional Tensor) - shape (...), dtype float32

等价实现

```

def ref_residual_layer_norm_smoothquant(input, residual, weight, bias, smooth_scales,
    ↪ i8_output=None, output_scales=None, eps=1e-5):
    residual_out = input + residual

    input = torch.nn.functional.layer_norm(
        residual_out, [weight.shape[-1]], weight, bias, eps=eps
    )

```

```

i8_output, output_scales = ref_dynamic_scaled_quant_smoothquant(input, smooth_scales,
↪ i8_output, output_scales)

return residual_out, i8_output, output_scales

```

1.8.1.21 softmax

```
def softmax(input: torch.Tensor, dim=None, _stacklevel=3, dtype=None)
```

参数说明

该模块支持 float16

- input - input
- dim - 要进行 softmax 的维度, 目前只支持最后一维, dim==-1 or dim == input.dim()-1
- _stacklevel - 这个参数只是为了与 pytorch 中对齐, stacklevel is used in python to indicate warning mechanism how far up the stack it has to go to find the line that called the function which issued the warning. For example, the code below makes the warning refer to deprecation()'s caller by using stacklevel=2, rather than to the source of deprecation() itself. stacklevel=3 would refer to the caller of deprecation()'s caller and so on

```
def deprecation(message):
    warnings.warn(message, DeprecationWarning, stacklevel=2)
```

- dtype - the desired data type of returned tensor

返回值

- Tensor - same shape as input

等价实现

```
def ref_softmax(input: torch.Tensor, dim: int = None, _stacklevel: int = 3, dtype=None):
    out = torch.nn.functional.softmax(
        input, dim=dim, _stacklevel=_stacklevel, dtype=dtype
    )
    return out

```

1.8.1.22 reshape_and_cache

同 `vllm_reshape_and_cache_v4`

1.8.1.23 reshape_and_cache_v5

同 `vllm_cache_ops_reshape_and_cache`

1.8.1.24 vllm_llama_mlp

```
class vllm_llama_mlp(CF.VllmLlamaMlp):
    def __init__(
        self,
        gate_up_proj_weight: "torch.Tensor",
        down_proj_weight: "torch.Tensor",
        hidden_size: int,
        intermediate_size: int,
        tp: int,
    ) -> None:
        self.gate_up_proj_weight = gate_up_proj_weight
        self.down_proj_weight = down_proj_weight
        self.hidden_size = hidden_size
        self.intermediate_size = intermediate_size
        self.tp = tp
        super().__init__(
            self.gate_up_proj_weight,
            self.down_proj_weight,
            self.hidden_size,
            self.intermediate_size,
            self.tp,
        )

    def __call__(self, x: "torch.Tensor", group=None):
        x1 = x
        if group is None:
            super().forward(x1, x1)
        else:
            super().forward(x1, x1, group)
        return x
```

参数说明

vllm_llama_mlp 是一个, 用于处理 MLP 部分

- gate_up_proj_weight - shape: (2 * intermediate_size / tp, hidden_size), dtype: float16
- down_proj_weight - shape: (hidden_size, intermediate_size / tp), dtype: float16
- hidden_size - int
- intermediate_size - int
- tp - int

1.8.2 ixformer.functions (融合算子)

1.8.2.1 silu_and_mul

```
ixformer.functions.silu_and_mul(input: torch.Tensor,
                                output: torch.Tensor = None) -> torch.Tensor
```

Pytorch 等价实现:

```
def ref_silu_and_mul(x: torch.Tensor) -> torch.Tensor:
    x1, x2 = x.chunk(chunks=2, dim=-1)
    return F.silu(x1) * x2
```

参数说明

- Shape:
 - input: [*,2c]
 - output: [*,c]
- 支持数据类型: bfloat16, float16, float32

1.8.2.2 gelu_and_mul

```
ixformer.functions.gelu_and_mul(input: torch.Tensor,
                                 output: torch.Tensor = None,
                                 gate_first=True) -> torch.Tensor
```

Pytorch 等价实现:

```
import torch.nn.functional as NNF
def ref_gelu_and_mul(input: "torch.Tensor", gate_first=True) -> torch.Tensor:
    x1, x2 = input.chunk(chunks=2, dim=-1)
    if gate_first:
        res = NNF.gelu(x1) * x2
    else:
        res = NNF.gelu(x2) * x1
    return res
```

参数说明

- Shape:
 - input: [*,2c]
 - output: [*,c]
 - gate_first: 判断是否先执行 gate
- 支持数据类型: bfloat16, float16, float32

1.8.2.3 gelu_tanh_and_mul

```
ixformer.functions.gelu_tanh_and_mul(input: torch.Tensor,
                                     output: torch.Tensor = None) -> torch.Tensor
```

Pytorch 等价实现:

```
import torch.nn.functional as NNF
def ref_gelu_tanh_and_mul(input: "torch.Tensor") -> torch.Tensor:
    x1, x2 = input.chunk(chunks=2, dim=-1)
    res = NNF.gelu(x1) * x2
    return res
```

参数说明

- Shape:
 - input: $[*, 2c]$
 - output: $[*, c]$
- 支持数据类型: bfloat16, float16, float32

1.8.2.4 rms_norm

```
ixformer.functions.rms_norm(input: torch.Tensor,
                             weight: torch.Tensor,
                             eps: float = 1e-6,
                             output: torch.Tensor = None) -> torch.Tensor
```

Pytorch 等价实现:

```
def ref_rms_norm(
    input: torch.Tensor,
    weight: torch.Tensor,
    eps: float = 1e-6,
):
    dtype = input.dtype
    input = input.float()
    weight = weight.float()
    output = input * torch.rsqrt(input.pow(2).mean(-1, keepdim=True) + eps)
    output = output * weight
    return output.to(dtype)
```

参数说明

- Shape:
 - input: $[*, c]$
 - weight: $[c]$
 - eps: a value added to the denominator for numerical stability. Default: 1e-6
 - output: $[*, c]$
- 支持数据类型: bfloat16, float16

1.8.2.5 act_bias_mm

```
ixformer.functions.act_bias_mm(
    mat1: torch.Tensor,
    mat2: torch.Tensor,
    bias: torch.Tensor = None,
    output: torch.Tensor = None,
    scale: float = 1,
    act_type: str = "none",
    trans_format: str = "NN",
) -> torch.Tensor
```

Pytorch 等价实现:

模式1: gemm, 没有 activation 及 bias, output = (mat1@mat2)*scale

模式2: gemm + bias, output = (mat1@mat2)*scale + bias

模式3: gemm + bias + activation, output = activation((mat1@mat2)*scale + bias)

参数说明

- mat1: float16 [* ,m,k]
- mat2: float16 [* ,n,k] if "TN" else [* ,k,n]
- bias: float16 [n]
- output: float16 [* ,m,n]
- act_type: "none"、"gelu"、"relu"、"silu"
- trans_format: "TN" 或 "NN"

1.8.2.6 fused_group_norm_silu

```
ixformer.functions.fused_group_norm_silu(
    input: torch.Tensor,
    num_groups: int,
    weight: torch.Tensor,
    bias: torch.Tensor,
    eps: float = 1e-05,
    is_nhwc: bool = False, #
    act_type: int=1,
) -> torch.Tensor
```

Pytorch 等价实现:

```
def ref_fused_group_norm_silu(
    input: torch.Tensor,
    num_groups: int,
    weight: torch.Tensor,
    bias: torch.Tensor,
    eps: float = 1e-05,
```

```

):
    output = torch.nn.functional.group_norm(input, num_groups, weight, bias, eps)
    output = output * torch.sigmoid(output)
    return output

```

参数说明

- Shape:
 - input: "contiguous_format": [n,c,h,w] or [n,c,h] "channels_last": [n,h,w,c]
 - num_groups: int, number of groups to separate the channels into
 - weight: [c], affine parameters
 - bias: [c], affine parameters
 - eps: a value added to the denominator for numerical stability. Default: 1e-5
 - is_nhw: Determine whether the input shape is [n,h,w,c]
 - act_type: 0 or 1, if act_type=1, silu
- 支持数据类型: float16

1.8.2.7 dynamic_scaled_quant_smoothquant

```

ixformer.functions.dynamic_scaled_quant_smoothquant(
    input: "torch.Tensor",
    smooth_scales: "torch.Tensor",
    i8_output: "torch.Tensor"=None,
    output_scales: "torch.Tensor"=None,
) -> torch.Tensor

```

Pytorch 等价实现:

```

def ref_dynamic_scaled_quant_smoothquant(input, smooth_scales, i8_output=None,
    ↪ output_scales=None):
    scales_shape = input.shape[:-1]
    input = input.float()
    output = input * smooth_scales.view(1, -1)
    amax_, _ = torch.max(torch.abs(output), dim=-1, keepdim=True)
    scales = amax_ / 127.0
    output = output / scales
    output = torch.clamp(torch.round(output), -127, 127).to(torch.int8)
    if i8_output is not None:
        i8_output.copy_(output)
        output = i8_output
    if output_scales is not None:
        output_scales.view(-1).copy_(scales.view(-1))
        scales = output_scales

    return output, scales.view(scales_shape)

```

参数说明

- Shape:
 - input: [...,K], dtype: float16,bfloat16
 - smooth_scales: [K], dtype: float16,bfloat16
 - i8_output: [...,K], dtype: int8
 - output_scales: [...], dtype: float32
- 支持数据类型: float16, bfloat16

1.8.2.8 residual_layer_norm

```
ixformer.functions.residual_layer_norm(
    input: torch.Tensor,
    normalized_shape: int,
    weight: torch.Tensor,
    bias: torch.Tensor,
    residual: torch.Tensor,
    residual_bias: torch.Tensor = None,
    alpha: float = 1.0,
    eps: float = 1e-5,
    is_post_ln=False,
)-> torch.Tensor
```

Pytorch 等价实现:

```
def ref_residual_layer_norm(
    input: torch.Tensor,
    normalized_shape: int,
    weight: torch.Tensor,
    bias: torch.Tensor,
    residual: torch.Tensor,
    residual_bias: torch.Tensor = None,
    alpha: float = 1.0,
    eps: float = 1e-5,
    is_post_ln=False,
):
    dtype = input.dtype
    if residual_bias is None:
        x = input.float() + residual.float() * alpha
    else:
        x = input.float() + residual.float() * alpha + residual_bias.float()

    y = torch.nn.functional.layer_norm(
        x.to(dtype), normalized_shape, weight, bias, eps=eps
    )

    if is_post_ln:
        return y, y
    else:
        return y, x.to(dtype)
```

参数说明

- Shape:
 - input: [batch_count * seq_len, hidden_size]
 - normalized_shape: int = input.shape[-1]
 - residual: [batch_count * seq_len, hidden_size]
 - residual_bias: [hidden_size]
 - weight: [hidden_size]
 - bias: [hidden_size]
 - alpha: float = 1.0
 - eps: a value added to the denominator for numerical stability. Default: 1e-5
 - is_post_ln: bool, post-layernorm or pre-layernorm
- 支持数据类型: float16, bfloat16

1.8.2.9 fused_addmm_bias_col_act

```
ixformer.functions.fused_addmm_bias_col_act(
    input:torch.Tensor,
    mat1:torch.Tensor,
    mat2:torch.Tensor,
    *,
    beta: float =1,
    alpha: float =1,
    out:torch.Tensor=None,
    bias:torch.Tensor=None,
    activation:Union[int, str] = 2)-> torch.Tensor
```

Pytorch 等价实现:

```
def ref_fused_addmm_bias_col_act(
    input, mat1, mat2, *, beta=1, alpha=1, out=None, bias=None, activation=2
):
    if isinstance(activation, int):
        assert activation in id_to_activation
    if isinstance(activation, str):
        assert activation in activation_to_id
        activation = activation_to_id[activation]
    if activation == 2:
        output_pt = torch.addmm(input, mat1, mat2, alpha=alpha, beta=beta) + bias
    elif activation == 3:
        output_pt = torch.nn.functional.gelu(
            torch.addmm(input, mat1, mat2, alpha=alpha, beta=beta) + bias
        )
    else:
        output_pt = torch.nn.functional.relu(
            torch.addmm(input, mat1, mat2, alpha=alpha, beta=beta) + bias
        )
    return output_pt
```

参数说明

- Shape:
 - input: [n,p], matrix to be added
 - imat1: [n,m], the first matrix to be matrix multiplied
 - imat2: [m,p], the second matrix to be matrix multiplied
 - ibeta: multiplier for input
 - ialpha: multiplier for `mat1@mat2`
 - iout: [n,p]
 - ibias: [n,p]
 - iactivation: str or int, activation function
- 支持数据类型: float16, bfloat16, float32

1.8.2.10 residual_rms_norm

```
ixformer.functions.residual_rms_norm(
    input: torch.Tensor,
    residual: torch.Tensor,
    weight: torch.Tensor,
    eps: float = 1e-5,
) -> torch.Tensor
```

Pytorch 等价实现:

```
def ref_residual_rms_norm(
    input: torch.Tensor,
    residual: torch.Tensor,
    weight: torch.Tensor,
    eps: float = 1e-5,
):
    dtype = input.dtype
    residual_output = input + residual
    input = input.float() + residual.float()
    weight = weight.float()
    output = input * torch.rsqrt(input.pow(2).mean(-1, keepdim=True) + eps)
    output = output * weight
    return output.to(dtype), residual_output
```

参数说明

- Shape:
 - input: [N,*]
 - residual: [N,*]
 - residual_bias: [hidden_size]
 - weight: [input.shape[-1]]
 - eps: a value added to the denominator for numerical stability. Default: 1e-5
- 支持数据类型: float16, bfloat16

1.8.2.11 layer_norm_2sb_fused

```
ixformer.functions.layer_norm_2sb_fused(
    input: torch.Tensor,
    normalized_shape: List[int],
    ln_weight1: torch.Tensor,
    ln_bias1: torch.Tensor,
    ln_weight2: torch.Tensor,
    ln_bias2: torch.Tensor,
    eps: float = 1e-5,
)-> torch.Tensor
```

Pytorch 等价实现:

```
def ref_layer_norm_2sb_fused(
    input: torch.Tensor,
    normalized_shape: List[int],
    ln_weight1: torch.Tensor,
    ln_bias1: torch.Tensor,
    ln_weight2: torch.Tensor,
    ln_bias2: torch.Tensor,
    eps: float = 1e-5,
):
    assert input.shape[-1] <= 16384
    # normalized_shape 需要是 list 或者 tuple, 并且不能为空
    norm_size = 1
    if normalized_shape == None:
        norm_size = ln_weight1.size(-1)
        normalized_shape = [norm_size]
    else:
        if (
            isinstance(normalized_shape, list) or isinstance(normalized_shape, tuple)
        ) and len(normalized_shape) >= 1:
            for i in normalized_shape:
                norm_size = i * norm_size
        else:
            raise f"layer_norm(): argument 'normalized_shape' (position 2) must be tuple
                of ints, not {type(normalized_shape)}"

    if torch.numel(input) % norm_size != 0:
        raise f"Given normalized_shape={normalized_shape}, expected input with shape [*,"
            f"{norm_size}], but got input of {input.shape}"

    output1 = torch.nn.functional.layer_norm(
        input, normalized_shape, ln_weight1, ln_bias1, eps=eps
    )

    output2 = torch.nn.functional.layer_norm(
        input, normalized_shape, ln_weight2, ln_bias2, eps=eps
    )
```

```
return output1.to(input.dtype), output2.to(input.dtype)
```

参数说明

- Shape:
 - input: [batch_count * seq_len, hidden_size]
 - normalized_shape: List[int] = [hidden_size]
 - ln_weight1: [hidden_size]
 - ln_bias1: [hidden_size]
 - ln_weight2: [hidden_size]
 - ln_bias2: [hidden_size]
 - eps: a value added to the denominator for numerical stability. Default: 1e-5
- 支持数据类型: float16, bfloat16

1.8.2.12 scaled_dot_product_attention

```
ixformer.functions.scaled_dot_product_attention(
    query: "torch.Tensor",
    key: "torch.Tensor",
    value: "torch.Tensor",
    attn_mask=None,
    dropout_p=0.0,
    is_causal=False,
)-> torch.Tensor
```

Pytorch 等价实现:

```
def ref_scaled_dot_product_attention(
    query: "torch.Tensor",
    key: "torch.Tensor",
    value: "torch.Tensor",
    attn_mask=None,
    dropout_p=0.0,
    is_causal=False,
):
    assert len(query.shape) >= 3
    assert len(key.shape) >= 3
    assert len(value.shape) >= 3
    batch_size = query.shape[0]
    L = query.shape[-2]
    S = key.shape[-2]

    if is_causal and attn_mask is not None:
        raise RuntimeError()

    if attn_mask is None and is_causal is False:
        attn_mask = torch.ones([batch_size, 1, 1, S]).bool().to(query.device)
```

```

if is_causal:
    attn_mask = torch.ones(L, S, dtype=torch.bool).tril(diagonal=0).to(query.device)
if attn_mask.dtype == torch.bool:
    attn_mask = (
        torch.zeros_like(attn_mask).to(query.dtype).masked_fill(~attn_mask, -10000)
    )
# attn_mask = torch.ones(L, S, dtype=torch.bool).tril(diagonal=0) if is_causal else
# attn_mask = attn_mask.masked_fill(not attn_mask, -float('inf')) if
attn_weight = torch.softmax(
    (query @ key.transpose(-2, -1) / math.sqrt(query.size(-1))) + attn_mask, dim=-1
)
attn_weight = torch.dropout(attn_weight, dropout_p, True)
return attn_weight.to(query.dtype) @ value

```

参数说明

- Shape:
 - query: [N, ..., L, E]
 - key: [N, ..., S, E]
 - value: [N, ..., S, Ev]
 - attn_mask: [(N, ..., L, S)]
 - dropout_p: float, Dropout probability; if greater than 0.0, dropout is applied
 - is_causal: bool, If true, assumes causal attention masking and errors if both attn_mask and is_causal are set.
- 支持数据类型: float16, bfloat16

1.8.2.13 silu_and_mul_smoothquant

```

ixformer.functions.silu_and_mul_smoothquant(
    input: "torch.Tensor",
    smooth_scales: "torch.Tensor",
    i8_output: "torch.Tensor"=None,
    output_scales: "torch.Tensor"=None,
)-> torch.Tensor

# Pytorch 等价实现:
def ref_silu_and_mul_smoothquant(input, smooth_scales, i8_output=None,
    output_scales=None):
    x1, x2 = input.chunk(chunks=2, dim=-1)
    x = NNF.silu(x1) * x2

    return ref_dynamic_scaled_quant_smoothquant(x, smooth_scales, i8_output, output_scales)
def ref_dynamic_scaled_quant_smoothquant(input, smooth_scales, i8_output=None,
    output_scales=None):

```

```
# input: bf/fp16 [...,k]
# smooth_scales: bf/fp16 [k]
scales_shape = input.shape[:-1]
input = input.float()
output = input * smooth_scales.view(1, -1)
amax_, _ = torch.max(torch.abs(output), dim=-1, keepdim=True)
scales = amax_ / 127.0
output = output / scales
output = torch.clamp(torch.round(output), -127, 127).to(torch.int8)
if i8_output is not None:
    i8_output.copy_(output)
    output = i8_output
if output_scales is not None:
    output_scales.view(-1).copy(scales.view(-1))
    scales = output_scales

return output, scales.view(scales_shape)
```

参数说明

- Shape:
 - input: bf/fp16, [...,2*K]
 - smooth_scales: bf/fp16, [K]
 - i8_output: int8, [...,K]
 - output_scales: float32, [...]
- 支持数据类型: float16, bfloat16

1.8.2.14 rms_norm_smoothquant

```
ixformer.functions.rms_norm_smoothquant(
    input: "torch.Tensor",
    weight: "torch.Tensor",
    smooth_scales: "torch.Tensor",
    i8_output: "torch.Tensor"=None,
    output_scales: "torch.Tensor"=None,
    eps=1e-5
)-> torch.Tensor
```

Pytorch 等价实现:

```
def ref_rms_norm_smoothquant(input, weight, smooth_scales, i8_output=None,
    ↪ output_scales=None, eps=1e-5):
    dtype = input.dtype
    input = input.float()
    weight = weight.float()
    input = input * torch.rsqrt(input.pow(2).mean(-1, keepdim=True) + eps)
    input = (input * weight).to(dtype)
```

```

return ref_dynamic_scaled_quant_smoothquant(input, smooth_scales, i8_output,
    ↪ output_scales)

def ref_dynamic_scaled_quant_smoothquant(input, smooth_scales, i8_output=None,
    ↪ output_scales=None):
    # input: bf/fp16 [...,k]
    # smooth_scales: bf/fp16 [k]
    scales_shape = input.shape[:-1]
    input = input.float()
    output = input * smooth_scales.view(1, -1)
    amax_, _ = torch.max(torch.abs(output), dim=-1, keepdim=True)
    scales = amax_ / 127.0
    output = output / scales
    output = torch.clamp(torch.round(output), -127, 127).to(torch.int8)
    if i8_output is not None:
        i8_output.copy_(output)
        output = i8_output
    if output_scales is not None:
        output_scales.view(-1).copy(scales.view(-1))
        scales = output_scales

    return output, scales.view(scales_shape)

```

参数说明

- Shape:
 - input: bf/fp16, [...,K]
 - weight: bf/fp16, [K]
 - smooth_scales: bf/fp16, [K]
 - i8_output: int8, [...,K]
 - output_scales: float32, [...]
 - eps: a value added to the denominator for numerical stability. Default: 1e-5
- 支持数据类型: float16, bfloat16

1.8.2.15 residual_rms_norm_smoothquant

```

ixformer.functions.residual_rms_norm_smoothquant(
    input: "torch.Tensor",
    residual: "torch.Tensor",
    weight: "torch.Tensor",
    smooth_scales: "torch.Tensor",
    i8_output: "torch.Tensor"=None,
    output_scales: "torch.Tensor"=None,
    eps=1e-5
)-> torch.Tensor

```

Pytorch 等价实现:

```
def ref_residual_rms_norm_smoothquant(input, residual, weight, smooth_scales,
    i8_output=None, output_scales=None, eps=1e-5):
    dtype = input.dtype
    residual_out = input + residual
    input = residual_out.float()
    weight = weight.float()
    input = input * torch.rsqrt(input.pow(2).mean(-1, keepdim=True) + eps)
    input = (input * weight).to(dtype)
    i8_output, output_scales = ref_dynamic_scaled_quant_smoothquant(input, smooth_scales,
    i8_output, output_scales)

    return i8_output, residual_out, output_scales

def ref_dynamic_scaled_quant_smoothquant(input, smooth_scales, i8_output=None,
    output_scales=None):
    # input: bf/fp16 [...,k]
    # smooth_scales: bf/fp16 [k]
    scales_shape = input.shape[:-1]
    input = input.float()
    output = input * smooth_scales.view(1, -1)
    amax_, _ = torch.max(torch.abs(output), dim=-1, keepdim=True)
    scales = amax_ / 127.0
    output = output / scales
    output = torch.clamp(torch.round(output), -127, 127).to(torch.int8)
    if i8_output is not None:
        i8_output.copy_(output)
        output = i8_output
    if output_scales is not None:
        output_scales.view(-1).copy(scales.view(-1))
        scales = output_scales

    return output, scales.view(scales_shape)
```

参数说明

- Shape:
 - input: bf/fp16, [...,K]
 - residual: bf/fp16, [...,K]
 - weight: bf/fp16, [K]
 - smooth_scales: bf/fp16, [K]
 - i8_output: int8, [...,K]
 - output_scales: float32, [...]
 - eps: a value added to the denominator for numerical stability. Default: 1e-5
- 支持数据类型: float16, bfloat16

1.8.2.16 layer_norm_2sb_smoothquant

```
ixformer.functions.layer_norm_2sb_smoothquant(
    input: "torch.Tensor",
    weight1: "torch.Tensor",
    bias1: "torch.Tensor",
    smooth_scales1: "torch.Tensor",
    weight2: "torch.Tensor",
    bias2: "torch.Tensor",
    smooth_scales2: "torch.Tensor",
    i8_output1: "torch.Tensor"=None,
    output_scales1: "torch.Tensor"=None,
    i8_output2: "torch.Tensor"=None,
    output_scales2: "torch.Tensor"=None,
    eps=1e-5
)-> torch.Tensor
# Pytorch 等价实现:
def ref_layer_norm_2sb_smoothquant(input, weight1, bias1, smooth_scales1,
                                   weight2, bias2, smooth_scales2,
                                   i8_output1=None, output_scales1=None,
                                   i8_output2=None, output_scales2=None,
                                   eps=1e-5):
    input1 = torch.nn.functional.layer_norm(
        input, [weight1.shape[-1]], weight1, bias1, eps=eps
    )
    input2 = torch.nn.functional.layer_norm(
        input, [weight2.shape[-1]], weight2, bias2, eps=eps
    )

    i8_output1, output_scales1 = ref_dynamic_scaled_quant_smoothquant(input1,
        ↪ smooth_scales1, i8_output1, output_scales1)
    i8_output2, output_scales2 = ref_dynamic_scaled_quant_smoothquant(input2,
        ↪ smooth_scales2, i8_output2, output_scales2)

    return i8_output1, output_scales1, i8_output2, output_scales2
def ref_dynamic_scaled_quant_smoothquant(input, smooth_scales, i8_output=None,
    ↪ output_scales=None):
    # input: bf/fp16 [...,k]
    # smooth_scales: bf/fp16 [k]
    scales_shape = input.shape[:-1]
    input = input.float()
    output = input * smooth_scales.view(1, -1)
    amax_, _ = torch.max(torch.abs(output), dim=-1, keepdim=True)
    scales = amax_ / 127.0
    output = output / scales
    output = torch.clamp(torch.round(output), -127, 127).to(torch.int8)
    if i8_output is not None:
        i8_output.copy_(output)
        output = i8_output
    if output_scales is not None:
```

```

output_scales.view(-1).copy(scales.view(-1))
scales = output_scales

return output, scales.view(scales_shape)

```

参数说明

- Shape:
 - input: bf/fp16, [...,K]
 - weight1: bf/fp16,[K]
 - bias1: [K]
 - smooth_scales1: [K]
 - weight2: bf/fp16, [K]
 - bias2: [K]
 - smooth_scales2: [K]
 - i8_output1: int8, [...,K]
 - output_scales1: float32, [...]
 - i8_output2: int8, [...,K]
 - output_scales2: float32, [...]
 - eps: a value added to the denominator for numerical stability. Default: 1e-5
- 支持数据类型: float16, bfloat16

1.8.2.17 residual_layer_norm_2sb_smoothquant

```

ixformer.functions.residual_layer_norm_2sb_smoothquant(
    input: "torch.Tensor",
    residual: "torch.Tensor",
    weight1: "torch.Tensor",
    bias1: "torch.Tensor",
    smooth_scales1: "torch.Tensor",
    weight2: "torch.Tensor",
    bias2: "torch.Tensor",
    smooth_scales2: "torch.Tensor",
    i8_output1: "torch.Tensor"=None,
    output_scales1: "torch.Tensor"=None,
    i8_output2: "torch.Tensor"=None,
    output_scales2: "torch.Tensor"=None,
    eps=1e-5
) -> torch.Tensor

```

Pytorch 等价实现:

```

def ref_residual_layer_norm_2sb_smoothquant(input, residual,
                                             weight1, bias1, smooth_scales1,
                                             weight2, bias2, smooth_scales2,
                                             i8_output1=None, output_scales1=None,
                                             i8_output2=None, output_scales2=None,

```

```

                                eps=1e-5):
    residual_out = input + residual
    input1 = torch.nn.functional.layer_norm(
        residual_out, [weight1.shape[-1]], weight1, bias1, eps=eps
    )
    input2 = torch.nn.functional.layer_norm(
        residual_out, [weight2.shape[-1]], weight2, bias2, eps=eps
    )

    i8_output1, output_scales1 = ref_dynamic_scaled_quant_smoothquant(input1,
↪ smooth_scales1, i8_output1, output_scales1)
    i8_output2, output_scales2 = ref_dynamic_scaled_quant_smoothquant(input2,
↪ smooth_scales2, i8_output2, output_scales2)

    return residual_out, i8_output1, output_scales1, i8_output2, output_scales2
def ref_dynamic_scaled_quant_smoothquant(input, smooth_scales, i8_output=None,
↪ output_scales=None):
    # input: bf/fp16 [...,k]
    # smooth_scales: bf/fp16 [k]
    scales_shape = input.shape[:-1]
    input = input.float()
    output = input * smooth_scales.view(1, -1)
    amax_, _ = torch.max(torch.abs(output), dim=-1, keepdim=True)
    scales = amax_ / 127.0
    output = output / scales
    output = torch.clamp(torch.round(output), -127, 127).to(torch.int8)
    if i8_output is not None:
        i8_output.copy_(output)
        output = i8_output
    if output_scales is not None:
        output_scales.view(-1).copy(scales.view(-1))
        scales = output_scales

    return output, scales.view(scales_shape)

```

参数说明

- Shape:
 - input: bf/fp16, [...,K]
 - residual: bf/fp16, [...,K]
 - weight1: bf/fp16, [K]
 - bias1: [K]
 - smooth_scales1: [K]
 - weight2: bf/fp16, [K]
 - bias2: [K]
 - smooth_scales2: [K]
 - i8_output1: int8, [...,K]
 - output_scales1: float32, [...]
 - i8_output2: int8, [...,K]

- output_scales2: float32, [...]
- eps: a value added to the denominator for numerical stability. Default: 1e-5
- 支持数据类型: float16, bfloat16

1.8.2.18 lightllm_apply_penalty

```
ixformer.functions.lightllm_apply_penalty(
    Logits: torch.Tensor,
    presence_penalty: torch.Tensor,
    frequency_penalty: torch.Tensor,
    p_token_ids: torch.Tensor,
    p_token_counts: torch.Tensor,
    p_cumsum_seq_len: torch.Tensor,
    p_max_len_in_batch: int,
) -> torch.Tensor

# Pytorch 等价实现:
def ref_lightllm_apply_penalty(
    Logits: torch.Tensor,
    presence_penalty: torch.Tensor,
    frequency_penalty: torch.Tensor,
    p_token_ids: torch.Tensor,
    p_token_counts: torch.Tensor,
    p_cumsum_seq_len: torch.Tensor,
    p_max_len_in_batch: int,
):
    batch_size = Logits.size(0)
    output = Logits.clone()
    for cur_batch in range(batch_size):
        cur_frequency = frequency_penalty[cur_batch]
        cur_presence = presence_penalty[cur_batch]
        cur_batch_start_index = p_cumsum_seq_len[cur_batch]
        cur_batch_end_index = p_cumsum_seq_len[cur_batch + 1]
        for token_idx in range(cur_batch_start_index, cur_batch_end_index):
            batch_ids = p_token_ids[token_idx]
            batch_ids_count = p_token_counts[token_idx]
            cur_logits = output[cur_batch][batch_ids]

            freq_logits = cur_logits - batch_ids_count * cur_frequency
            pre_logits = freq_logits - cur_presence
            # if token_idx==0:
            #     print(f"batch_ids {batch_ids} cur_logits {cur_logits} pre_logits
            #           {pre_logits}")
            output[cur_batch][batch_ids] = pre_logits
    return output
```

参数说明

- Logits: [batch_size, vocab_size], dtype:[torch.float]

- presence_penalty: [batch_size], dtype:[torch.float]
- frequency_penalty: [batch_size], dtype:[torch.float]
- p_token_ids: [num_tokens], dtype:[torch.int32]
- p_token_counts: [num_tokens], dtype:[torch.int32]
- p_cumsum_seq_len: [batch_size+1], dtype:[torch.int32]
- p_max_len_in_batch: int, 在一个 batch 中 seq 的最大长度

支持数据类型: float32

1.8.2.19 lightllm_destindex_copy_kv

```
ixformer.functions.lightllm_destindex_copy_kv(
    key_cache: torch.Tensor,
    mem_idx: torch.Tensor,
    output: torch.Tensor,
) -> torch.Tensor
```

Pytorch 等价实现:

```
def ref_lightllm_destindex_copy_kv(
    key_cache: torch.Tensor,
    mem_idx: torch.Tensor,
    output: torch.Tensor,
):
    output[mem_idx.long()] = key_cache
    return output
```

参数说明

- key_cache: [tokens, num_kv_heads, head_size], dtype:[torch.half]
- mem_idx: [tokens], dtype:[torch.int32]
- output: [max_tokens, num_kv_heads, head_size], dtype:[torch.half]

支持数据类型: float16

1.8.2.20 lightllm_glm2_rope

```
ixformer.functions.lightllm_glm2_rope(
    x: torch.Tensor, # tokens, head_num, head_dim
    cos: torch.Tensor, # tokens, rot_dim
    sin: torch.Tensor,
) -> torch.Tensor
```

Pytorch 等价实现:

```
def ref_lightllm_glm2_rope(
    x: torch.Tensor, # tokens, head_num, head_dim
    cos: torch.Tensor, # tokens, rot_dim
```

```

sin: torch.Tensor,
):
    num_tokens, _, rot_dim = list(cos.shape)
    head_num = x.shape[1]
    x12 = x[:, :, : rot_dim * 2]
    x3 = x[:, :, rot_dim * 2 :]
    x12 = x12.reshape(num_tokens, head_num, rot_dim, 2)
    x1 = x12[:, :, :, 0]
    x2 = x12[:, :, :, 1]

    # out0 = q0 * cos - q1 * sin
    # out1 = q0 * sin + q1 * cos
    # q1, q2 是沿着 head_dim 维度，交叉取值的

    q1 = x1 * cos - x2 * sin
    q2 = x2 * cos + x1 * sin

    q12 = torch.stack([q1, q2], dim=-1)
    q12 = q12.reshape(num_tokens, head_num, -1)
    x_pytorch = torch.cat([q12, x3], dim=-1)
    return x_pytorch

```

参数说明

- x: [num_tokens, head_num, head_dim], dtype:[torch.half]
- cos: [num_tokens,1,head_dim//2//2], dtype:[torch.half]
- sin: [num_tokens,1,head_dim//2//2], dtype:[torch.half]

支持数据类型：float16

1.8.2.21 lightllm_tokenattention

```

ixformer.functions.lightllm_tokenattention(
    query: torch.Tensor,
    key_cache: torch.Tensor,
    value_cache: torch.Tensor,
    reg_tokens: torch.Tensor,
    b_req_idx: torch.Tensor,
    b_seq_len: torch.Tensor,
    scale: float,
    max_context_len: int,
    partition: int,
    output: torch.Tensor,
)->torch.Tensor

```

Pytorch 等价实现：

```

def ref_lightllm_tokenattention(
    query: torch.Tensor,

```

```

key_cache: torch.Tensor,
value_cache: torch.Tensor,
reg_tokens: torch.Tensor,
b_req_idx: torch.Tensor,
b_seq_len: torch.Tensor,
scale: float,
max_context_len: int,
):
    batch_size, tp_q_head_num_, head_dim_ = query.shape
    tp_k_head_num_ = key_cache.size(-2)
    sm_scale = scale
    curbatch_max_context_len = max_context_len
    tmp_k = torch.zeros(
        (batch_size, tp_q_head_num_, curbatch_max_context_len, head_dim_),
        dtype=query.dtype,
        device="cuda",
    )
    tmp_v = torch.zeros(
        (batch_size, tp_q_head_num_, curbatch_max_context_len, head_dim_),
        dtype=query.dtype,
        device="cuda",
    )
    mask = torch.ones([batch_size, 1, 1, curbatch_max_context_len])

    kv_group_num = tp_q_head_num_ // tp_k_head_num_
    for cur_batch in range(batch_size):
        cur_batch_req_idx = b_req_idx[cur_batch]
        seq_len = b_seq_len[cur_batch]
        mask[cur_batch, :, :, :seq_len] = 0
        # print(f"cur_batch {cur_batch}")

        for seq_idx in range(seq_len):
            k_loc = reg_tokens[cur_batch_req_idx][seq_idx]
            # print(k_loc)
            for cur_head in range(tp_q_head_num_):
                cur_kv_head = cur_head // kv_group_num
                tmp_k[cur_batch, cur_head, seq_idx, :] = key_cache[k_loc][cur_kv_head]
                tmp_v[cur_batch, cur_head, seq_idx, :] = value_cache[k_loc][cur_kv_head]
    mask = mask.cuda()
    # batch_size, self.tp_q_head_num_, 1, max_len_in_batch
    attn_score = (
        torch.matmul(
            query.view(batch_size, tp_q_head_num_, 1, head_dim_),
            tmp_k.transpose(-1, -2),
        )
        * sm_scale
    )
    attn_score = attn_score + mask * -1000
    attn_score = torch.softmax(attn_score, dim=-1)
    # batch_size, self.tp_q_head_num_, 1, head_dim

```

```
py_out = torch.matmul(attn_score.to(query.dtype), tmp_v).view(
    batch_size, tp_q_head_num_, -1
)
return py_out
```

参数说明

- query: [batch_size, head_num, head_dim], dtype:[torch.half, torch.bfloat16]
- key_cache: [max_num_tokens, head_num_kv, head_dim], dtype:[torch.half, torch.bfloat16]
- value_cache: [max_num_tokens, head_num_kv, head_dim], dtype:[torch.half, torch.bfloat16]
- reg_tokens: [max_requset, max_tokens], dtype:[torch.int32]
- b_req_idx: [batch_size], dtype:[torch.int32]
- b_seq_len: [batch_size], dtype:[torch.int32]
- scale: float. The scaling of QK^T before applying softmax
- max_context_len: int, b_seq_len.max()
- partition: int
- output: [batch_size, head_num, head_dim], dtype:[torch.half, torch.bfloat16]

支持数据类型: float16 bfloat16

1.8.2.22 vllm_rotary_embedding

```
ixformer.functions.vllm_rotary_embedding(
    positions: torch.Tensor,
    query: torch.Tensor,
    key: torch.Tensor,
    head_size: int,
    cos_sin_cache: torch.Tensor,
    is_neox_style: bool = True,
) -> torch.Tensor
```

Pytorch 等价实现:

```
import torch.nn.functional as F
def ref_vllm_rotary_embedding(
    positions: torch.Tensor,
    query: torch.Tensor,
    key: torch.Tensor,
    head_size: int,
    cos_sin_cache: torch.Tensor,
    is_neox_style: bool = True,
):
    def _rotate_neox(x: torch.Tensor) -> torch.Tensor:
        x1 = x[..., :x.shape[-1] // 2]
        x2 = x[..., x.shape[-1] // 2:]
        return torch.cat((-x2, x1), dim=-1)

    def _rotate_gptj(x: torch.Tensor) -> torch.Tensor:
```

```

x1 = x[..., ::2]
x2 = x[..., 1::2]
x = torch.stack((-x2, x1), dim=-1)
return x.flatten(-2)

query = query.view(*query.shape[:-1], -1, head_size)
key = key.view(*key.shape[:-1], -1, head_size)

cos_sin = cos_sin_cache[positions]
cos, sin = cos_sin.chunk(2, dim=-1)
if is_neox_style:
    cos = cos.repeat(1, 1, 2).unsqueeze(-2)
    sin = sin.repeat(1, 1, 2).unsqueeze(-2)
else:
    cos = cos.repeat_interleave(2, dim=-1).unsqueeze(-2)
    sin = sin.repeat_interleave(2, dim=-1).unsqueeze(-2)

rotate_fn = _rotate_neox if is_neox_style else _rotate_gptj
query_rot = query * cos + rotate_fn(query) * sin
key_rot = key * cos + rotate_fn(key) * sin

query = query_rot.flatten(-2)
key = key_rot.flatten(-2)
return query, key

```

参数说明

- positions: int64, [num_tokens]
- query, key: float16, bfloat32, float32, [num_tokens, num_heads*head_size]
- cos_sin_cache: float16, bfloat32, float32, [max_position, head_size]
- head_size: int
- is_neox_style: bool

支持数据类型：float16, bfloat32, float32

1.8.2.23 vllm_moe_topk_softmax

```

ixformer.functions.vllm_moe_topk_softmax(
    topk_weights: "torch.Tensor",
    topk_ids: "torch.Tensor",
    token_expert_indicies: "torch.Tensor",
    gating_output: "torch.Tensor",
) -> torch.Tensor

# Pytorch 等价实现:
def torch_topk_softmax(inputs, topk):
    score = torch.softmax(inputs, dim=-1, dtype=torch.float32)
    topk_weights, topk_ids = torch.topk(score, topk)
    return topk_weights, topk_ids

```

参数说明

- topk_weights: float32, [num_tokens, topk]
- topk_ids: int32, [num_tokens, topk]
- token_expert_indicies: int32, [num_tokens, topk]
- gating_output: float32, [num_tokens, num_experts]

支持数据类型: float32

1.8.2.24 vllm_cache_ops_reshape_and_cache

```
ixformer.functions.vllm_cache_ops_reshape_and_cache(
    key: torch.Tensor,
    value: torch.Tensor,
    key_cache: torch.Tensor,
    value_cache: torch.Tensor,
    slot_mapping: torch.Tensor,
) -> torch.Tensor

# Pytorch 等价实现:
def ref_vllm_cache_ops_reshape_and_cache(
    key: torch.Tensor,
    value: torch.Tensor,
    key_cache: torch.Tensor,
    value_cache: torch.Tensor,
    slot_mapping: torch.Tensor,
):
    num_tokens, num_heads, head_size = key.shape
    x = 16 // torch.tensor([], dtype=key.dtype).element_size()
    block_size = key_cache.size(3)

    reshaped_key = key.reshape(num_tokens, num_heads, head_size // x, x)
    for i in range(num_tokens):
        block_idx = torch.div(slot_mapping[i], block_size, rounding_mode="floor")
        block_offset = slot_mapping[i] % block_size
        key_cache[block_idx, :, :, block_offset, :] = reshaped_key[i]
        value_cache[block_idx, :, :, block_offset] = value[i]
```

参数说明

- key, value: bfloat16, float16 [num_tokens, num_heads, head_size]
- key_cache: bfloat16, float16 [num_blocks, num_heads, head_size/x, block_size, x]
- value_cache: bfloat16, float16 [num_blocks, num_heads, block_size, head_size]
- slot_mapping: int32 [num_tokens]

支持数据类型: bfloat16, float16

1.8.2.25 vllm_reshape_and_cache

```
ixformer.functions.vllm_reshape_and_cache(
    key: torch.Tensor,
    value: torch.Tensor,
    key_cache: torch.Tensor,
    value_cache: torch.Tensor,
    slot_mapping: torch.Tensor,
) -> torch.Tensor

# Pytorch 等价实现:
def ref_vllm_reshape_and_cache(
    key: torch.Tensor,
    value: torch.Tensor,
    key_cache: torch.Tensor,
    value_cache: torch.Tensor,
    slot_mapping: torch.Tensor,
):
    num_tokens, _, _ = key.shape
    block_size = key_cache.size(2)

    for i in range(num_tokens):
        block_idx = torch.div(slot_mapping[i], block_size, rounding_mode="floor")
        block_offset = slot_mapping[i] % block_size
        key_cache[block_idx, :, block_offset, :] = key[i]
        value_cache[block_idx, :, block_offset, :] = value[i]
```

参数说明

- key, value: bfloat16, float16 [num_tokens, num_heads, head_size]
- key_cache: bfloat16, float16 [num_blocks, num_heads, block_size, head_size]
- value_cache: bfloat16, float16 [num_blocks, num_heads, block_size, head_size]
- slot_mapping: long, [num_tokens]

支持数据类型: bfloat16, float16

1.8.2.26 vllm_copy_blocks

```
ixformer.functions.vllm_copy_blocks(
    key: List[torch.Tensor],
    value: List[torch.Tensor],
    block_mapping: torch.Tensor,
) -> torch.Tensor

# Pytorch 等价实现:
def ref_vllm_copy_blocks(
    key_caches,
```

```
value_caches,
block_mapping,
):
    for k,v in zip(key_caches,value_caches):
        src = block_mapping[:,0]
        dst = block_mapping[:,1]
        k[dst] = k[src]
        v[dst] = v[src]
```

参数说明

- key, value: bfloat16, float16, float32 [(num_block, num_kv_heads, block_size, head_size)]
- block_mapping: long, [(num_tokens, 2)]

支持数据类型: bfloat16, float16, float32

1.8.2.27 vllm_swap_blocks

```
ixformer.functions.vllm_swap_blocks(
    key: torch.Tensor,
    value: torch.Tensor,
    block_mapping: torch.Tensor,
)->torch.Tensor
```

Pytorch 等价实现:

```
def ref_vllm_swap_blocks(src, dst, mapping):
    for item in mapping:
        src_idx = item[0]
        dst_idx = item[1]
        dst[dst_idx] = src[src_idx].to(dst.device)
```

参数说明

- key, value: bfloat16, float16, float32 [(num_block, num_kv_heads, block_size, head_size)]
- block_mapping: int64, [(num_tokens, 2)]

支持数据类型: bfloat16, float16, float32

1.8.2.28 vllm_batched_rotary_embedding

```
ixformer.functions.vllm_batched_rotary_embedding(
    positions: torch.Tensor,
    query: torch.Tensor,
    key: torch.Tensor,
    head_size: int,
    cos_sin_cache: torch.Tensor,
```

```

    is_neox_style: bool,
    rotary_dim: int,
    offsets: torch.Tensor
) -> torch.Tensor

# Pytorch 等价实现:
import torch.nn.functional as F
def ref_vllm_batched_rotary_embedding(
    positions: torch.Tensor,
    query: torch.Tensor,
    key: torch.Tensor,
    head_size: int,
    cos_sin_cache: torch.Tensor,
    is_neox_style,
    rotary_dim: int,
    offsets: torch.Tensor
):
    def _rotate_neox(x: torch.Tensor) -> torch.Tensor:
        x1 = x[..., :x.shape[-1] // 2]
        x2 = x[..., x.shape[-1] // 2:]
        return torch.cat((-x2, x1), dim=-1)

    def _rotate_gptj(x: torch.Tensor) -> torch.Tensor:
        x1 = x[..., ::2]
        x2 = x[..., 1::2]
        x = torch.stack((-x2, x1), dim=-1)
        return x.flatten(-2)

    query = query.view(*query.shape[:-1], -1, head_size)
    key = key.view(*key.shape[:-1], -1, head_size)

    query_rot = query[..., :rotary_dim]
    key_rot = key[..., :rotary_dim]
    if rotary_dim < head_size:
        query_pass = query[..., rotary_dim:]
        key_pass = key[..., rotary_dim:]

    cos_sin = cos_sin_cache[torch.add(positions, offsets)]
    cos, sin = cos_sin.chunk(2, dim=-1)
    if is_neox_style:
        # NOTE(woosuk): Here we assume that the positions tensor has the
        # shape [batch_size, seq_len].
        cos = cos.repeat(1, 1, 2).unsqueeze(-2)
        sin = sin.repeat(1, 1, 2).unsqueeze(-2)
    else:
        cos = cos.repeat_interleave(2, dim=-1).unsqueeze(-2)
        sin = sin.repeat_interleave(2, dim=-1).unsqueeze(-2)

    rotate_fn = _rotate_neox if is_neox_style else _rotate_gptj

```

```

query_rot = query_rot * cos + rotate_fn(query_rot) * sin
key_rot = key_rot * cos + rotate_fn(key_rot) * sin

if rotary_dim < head_size:
    query = torch.cat((query_rot, query_pass), dim=-1)
    key = torch.cat((key_rot, key_pass), dim=-1)
else:
    query = query_rot
    key = key_rot
query = query.flatten(-2)
key = key.flatten(-2)
return query, key

```

参数说明

- positions: [num_tokens], dtype:[torch.int64]
- query, key: [num_tokens, num_heads * head_size]
- head_size: int
- cos_sin_cache: [max_position, head_size]
- is_neox_style: bool
- rotary_dim: int, 需要添加位置编码的长度
- offsets: [num_tokens], dtype:torch.int64, lora 输入对应的偏移。

支持数据类型: bfloat16, float16, float32

1.8.2.29 reshape_and_cache_flash

```

ixformer.functions.reshape_and_cache_flash(
    key: torch.Tensor,
    value: torch.Tensor,
    key_cache: torch.Tensor,
    value_cache: torch.Tensor,
    slot_mapping: torch.Tensor,
    kv_cache_dtype: str,
    k_scale: float,
    v_scale: float,
) -> None

```

Pytorch 等价实现:

```

def ref_reshape_and_cache_flash(
    key: torch.Tensor,
    value: torch.Tensor,
    key_cache: torch.Tensor,
    value_cache: torch.Tensor,
    slot_mapping: torch.Tensor,
    kv_cache_dtype: str,
    k_scale: float,
    v_scale: float,

```

```

):
    num_tokens, _, _ = key.shape

    num_blocks, block_size, num_heads, head_size = key_cache.shape
    key_cache = key_cache.view(num_blocks, num_heads, block_size, head_size)

    value_cache = value_cache.view(num_blocks, num_heads, block_size, head_size)

    block_size = key_cache.size(2)

    for i in range(num_tokens):
        block_idx = torch.div(slot_mapping[i], block_size, rounding_mode="floor")
        block_offset = slot_mapping[i] % block_size
        key_cache[block_idx, :, block_offset, :] = key[i]
        value_cache[block_idx, :, block_offset, :] = value[i]

```

参数说明

- key: [num_tokens, num_heads, head_size], dtype: torch.half, torch.float, torch.bfloat16
- value: [num_tokens, num_heads, head_size], dtype: torch.half, torch.float, torch.bfloat16
- key_cache: [num_blocks, block_size, num_heads, head_size], dtype: torch.half, torch.float, torch.bfloat16
- value_cache: [num_blocks, block_size, num_heads, head_size], dtype: torch.half, torch.float, torch.bfloat16
- slot_mapping: [num_tokens], dtype: torch.long

支持数据类型: bfloat16, float16, float32

1.8.2.30 vllm_moe_align_block_size

```

ixformer.functions.vllm_moe_align_block_size(
    topk_ids: "torch.Tensor",
    num_experts: int,
    block_size: int,
    sorted_ids: "torch.Tensor",
    expert_ids: "torch.Tensor",
    num_tokens_post_pad: "torch.Tensor",
) -> None

```

PyTorch 等价实现, 参考 vLLM 官方测试代码 (https://github.com/vllm-project/vllm/blob/main/tests/kernels/test_moe.py)

参数说明

- topk_ids: [num_tokens, topk], dtype: torch.int32
- num_experts: int
- block_size: int
- sorted_ids: [topk_ids.numel() + num_experts * (block_size - 1)], dtype: torch.int32
- expert_ids: [topk_ids.numel() + num_experts], dtype: torch.int32

1.8.2.31 vllm_invoke_fused_moe_kernel

```
ixformer.functions.vllm_invoke_fused_moe_kernel(
    A: "torch.Tensor",
    B: "torch.Tensor",
    C: "torch.Tensor",
    topk_weight: "torch.Tensor",
    topk_ids: "torch.Tensor",
    sorted_token_ids: "torch.Tensor",
    expert_ids: "torch.Tensor",
    num_tokens_post_padded: "torch.Tensor",
    mul_routed_weight: bool,
    top_k: int,
    block_size_m: int,
) -> None

# PyTorch 等价实现, 参考 vLLM 官方测试代码 (https://github.com/vllm-project/vllm/blob/main/
↳ tests/kernels/test_moe.py)
```

参数说明

- A: [bs*seq, K] / [bs*seq*top_k, K], dtype: torch.half,torch.bfloat16
- B: [num_experts, N, K], dtype: torch.half,torch.bfloat16
- C: [bs*seq, top_k, N], dtype: torch.half,torch.bfloat16
- topk_weight: [bs*seq, topk], dtype:torch.float32
- topk_ids: (bs*seq, topk), dtype:torch.int32
- sorted_ids: [topk_ids.numel() + num_experts * (block_size - 1)], dtype:torch.int32
- expert_ids: [topk_ids.numel() + num_experts], dtype:torch.int32
- num_tokens_post_pad: [1], dtype:torch.int32
- mul_routed_weight: bool
- top_k: int
- block_size_m: int

1.8.2.32 t5_split_qkv

```
ixformer.functions.t5_split_qkv(qkv: "torch.Tensor", head_num: int, head_dim:
↳ int) -> torch.Tensor
```

Pytorch 等价实现:

```
import torch.nn.functional as F
def ref_t5_split_qkv(qkv: "torch.Tensor", head_num: int, head_dim: int):
    assert qkv.size(-1) == head_dim * head_num * 3
    batch_size, seq_len, _ = qkv.shape
    q, k, v = torch.chunk(qkv, 3, dim=-1)
    q = reshape_query(q, head_num, head_dim)
    k = reshape_query(k, head_num, head_dim)
```

```

    v = reshape_query(v, head_num, head_dim)
    return q, k, v
def reshape_query(query, head_num, head_dim):
    batch_size, seq_len, _ = query.shape
    query = query.view(batch_size, seq_len, head_num, head_dim)
    query = query.transpose(1, 2)
    return query

```

参数说明

- qkv: [batch_size, seq_len, head_dim * head_num * 3]
- head_num: int
- head_dim: int

支持数据类型: bfloat16, float16

1.8.2.33 t5_split_qkv_update_kv_cache

```

ixformer.functions.t5_split_qkv_update_kv_cache(
    qkv: "torch.Tensor",
    past_key: "torch.Tensor",
    past_value: "torch.Tensor",
    head_num: int,
    head_dim: int,
): -> torch.Tensor

```

Pytorch 等价实现:

```

def ref_t5_split_qkv_update_kv_cache(
    qkv: "torch.Tensor",
    past_key: "torch.Tensor",
    past_value: "torch.Tensor",
    head_num: int,
    head_dim: int,
):
    def reshape_query(query, head_num, head_dim):
        batch_size, seq_len, _ = query.shape
        query = query.view(batch_size, seq_len, head_num, head_dim)
        query = query.transpose(1, 2)
        return query
    assert qkv.size(-1) == head_dim * head_num * 3
    batch_size, seq_len, _ = qkv.shape
    q, k, v = torch.chunk(qkv, 3, dim=-1)
    q = reshape_query(q, head_num, head_dim)
    k = reshape_query(k, head_num, head_dim)
    v = reshape_query(v, head_num, head_dim)
    k = torch.cat([past_key, k], dim=2)
    v = torch.cat([past_value, v], dim=2)
    return q, k, v

```

参数说明

- qkv: [batch_size, 1, head_dim * head_num * 3]
- past_key: [batch_size, head_num, seq_len - 1, head_dim]
- past_value: [batch_size, head_num, seq_len - 1, head_dim]
- head_num: int
- head_dim: int

支持数据类型: bfloat16, float16

1.8.2.34 tgi_apply_rotary

```
ixformer.functions.tgi_apply_rotary(
    queries: List[torch.Tensor],
    cos: "torch.Tensor",
    sin: "torch.Tensor",
    outs: List[torch.Tensor] = None,
    is_neox_style: bool = True,
): -> torch.Tensor
```

Pytorch 等价实现:

```
def tgi_apply_rotary_emb_torch(
    x: "torch.Tensor",
    cos: "torch.Tensor",
    sin: "torch.Tensor",
    interleaved: bool = False,
):
    """
    x: (seqlen, num_heads, headdim)
    cos, sin: (seqlen, 1, rotary_dim / 2)
    interleaved: bool. 在 interleaved 的实现中, 对奇偶维度旋转需要将维度两两交错, 实现较为复杂。
    """
    ro_dim = cos.shape[-1] * 2
    assert ro_dim <= x.shape[-1]
    assert cos.shape == sin.shape
    if cos.dim() == 2:
        cos = cos.unsqueeze(1)
        sin = sin.unsqueeze(1)
    if interleaved:
        cos = cos.repeat_interleave(2, dim=-1)
        sin = sin.repeat_interleave(2, dim=-1)
    else:
        cos = cos.repeat(1, 1, 2)
        sin = sin.repeat(1, 1, 2)
    return torch.cat(
        [
            x[..., :ro_dim].float() * cos.float()
            + rotate_half(x[..., :ro_dim].float(), interleaved) * sin.float(),
            x[..., ro_dim:].float(),
        ]
    )
```

```
],
    dim=-1,
).to(x.dtype)
```

参数说明

- queries: List[torch.Tensor], 如果 len(queries)=1, 则 list 元素的 shape 为 (num_tokens, num_heads, head_size), 如果 len(queries)=2, 则 list 元素的 shape(num_tokens, num_heads, head_size// 2), dtype:[torch.half, torch.float, torch.bfloat16]
- cos: [max_position, 1, head_size //2], dtype:[torch.half, torch.float, torch.bfloat16]
- sin: [max_position, 1, head_size //2], dtype:[torch.half, torch.float, torch.bfloat16]
- is_neox_style: bool
- outs: List[torch.Tensor], 如果 len(outs)=1, 则 list 元素的 shape 为 (num_tokens, num_heads, head_size); 如果 len(outs)=2, 则 list 元素的 shape(num_tokens, num_heads, head_size// 2), dtype:[torch.half, torch.float, torch.bfloat16]

支持数据类型: bfloat16, float16, float32

1.8.2.35 tgi_gather_prefill_logprobs

```
ixformer.functions.tgi_gather_prefill_logprobs(
    logits: "torch.Tensor",
    prefill_tokens_indices: "torch.Tensor",
    output: "torch.Tensor" = None,
):->torch.Tensor

# Pytorch 等价实现:
def tgi_gather_prefill_logprobs(
    logits: "torch.Tensor",
    prefill_tokens_indices: "torch.Tensor",
    sin: "torch.Tensor",
    output: "torch.Tensor" = None,
):
    prefill_logprobs_tensor = torch.log_softmax(out, -1)
    output = torch.gather(
        prefill_logprobs_tensor, 1, prefill_tokens_indices.view(-1, 1)
    )
    return output
```

参数说明

- logits: [num_tokens, vocab_size], dtype:[torch.half, torch.bfloat16]
- prefill_tokens_indices: [tokens_indices], dtype:[torch.int]
- output: [tokens_indices, 1], dtype:[torch.half, torch.bfloat16]

支持数据类型: bfloat16, float16

1.8.2.36 vllm_reshape_and_cache_v4

```
ixformer.functions.vllm_reshape_and_cache_v4(
    key: torch.Tensor,
    value: torch.Tensor,
    key_cache: torch.Tensor,
    value_cache: torch.Tensor,
    slot_mapping: torch.Tensor,
) -> torch.Tensor

# Pytorch 等价实现:
def ref_vllm_reshape_and_cache_v4(
    key: torch.Tensor,
    value: torch.Tensor,
    key_cache: torch.Tensor,
    value_cache: torch.Tensor,
    slot_mapping: torch.Tensor,
):
    num_tokens, num_heads, head_size = key.shape
    x = 16 // torch.tensor([], dtype=key.dtype).element_size()
    block_size = key_cache.size(3)

    reshaped_key = key.reshape(num_tokens, num_heads, head_size // x, x)
    for i in range(num_tokens):
        block_idx = torch.div(slot_mapping[i], block_size, rounding_mode="floor")
        block_offset = slot_mapping[i] % block_size
        key_cache[block_idx, :, :, block_offset, :] = reshaped_key[i]
        value_cache[block_idx, :, :, block_offset] = value[i]
```

参数说明

- key: (num_tokens, num_heads, head_size), dtype:[torch.half, torch.float, torch.bfloat16]
- value: (num_tokens, num_heads, head_size), dtype:[torch.half, torch.float, torch.bfloat16]
- key_cache: (num_blocks, num_heads, head_size//8, block_size, 8), dtype:[torch.half, torch.float, torch.bfloat16]
- value_cache: (num_blocks, num_heads, head_size, block_size), dtype:[torch.half, torch.float, torch.bfloat16]
- slot_mapping:(num_tokens), dtype:[torch.long]

支持数据类型: bfloat16, float16, float32

1.8.2.37 reshape_and_cache_v3

```
ixformer.functions.reshape_and_cache_v3(
    key: "torch.Tensor",
    value: "torch.Tensor",
    key_cache: "torch.Tensor",
```

```

    value_cache: "torch.Tensor",
    slot_mapping: "torch.Tensor",
) -> torch.Tensor

# Pytorch 等价实现:
def ref_reshape_and_cache_v3(
    key,
    value,
    key_cache,
    value_cache,
    slot_mapping,
    num_tokens,
    num_heads,
    head_size,
    block_size,
):
    reshaped_key = key.view(num_tokens, num_heads, head_size // 32, 32)
    reshaped_value = value.reshape(num_tokens, num_heads, head_size // 32, 32)
    for i in range(num_tokens):
        block_idx = torch.div(slot_mapping[i], block_size, rounding_mode="floor")
        block_offset = slot_mapping[i] % block_size

        key_cache[
            block_idx, :, block_offset // 4, :, block_offset % 4, :
        ] = reshaped_key[i]
        value_cache[
            block_idx, :, block_offset // 4, :, block_offset % 4, :
        ] = reshaped_value[i]

```

参数说明

- 该函数实现将 key, vlaue 值存放在 key_cache, value_cache 对应 block index 的位置上
- 该模块支持 float16, bfloat16, 对 key_cache, value_cache 进行 in place 操作

返回值

无

1.8.2.38 vllm_gptq_shuffle

```

ixformer.functions.vllm_gptq_shuffle(
    qweights: "torch.Tensor",
    g_idx: "torch.Tensor",
    weight_bits: int
)

```

参数说明

- pack_factor: 一般为 8
- qweights: [ic // pack_factor, oc], dtype: torch.int32
- g_idx: [ic], dtype: torch.int32
- weight_bits: int

1.8.2.39 gptq_gemm

```
ixformer.functions.gptq_gemm(
    input: torch.Tensor,
    qweight: torch.Tensor,
    qzeros: torch.Tensor,
    scales: torch.Tensor,
    g_idx: torch.Tensor,
    use_exllama: bool,
    weight_bits: int
) -> torch.Tensor
```

参数说明

- pack_factor: 一般为 8
- input: [bs, ic], dtype:[torch.float16,torch.bfloat16]
- qweights: [ic // pack_factor, oc], dtype:torch.int32
- qzeros: [ic//group_size,oc//pack_factor], dtype:torch.int32
- scales: [ic // group_size, oc], dtype:[torch.float16,torch.bfloat16]
- g_idx: [ic], dtype:torch.int32
- use_exllama: bool
- weight_bits: int

1.8.2.40 flash_attn_varlen_func

```
ixformer.functions.flash_attn_varlen_func(
    q: torch.Tensor,
    k: torch.Tensor,
    v: torch.Tensor,
    cu_seqlens_q: torch.Tensor,
    cu_seqlens_k: torch.Tensor,
    max_seqlen_q: int,
    max_seqlen_k: int,
    dropout_p: float = 0.0,
    softmax_scale: float = None,
    causal: bool = False,
    return_attn_probs: bool = False,
    out: torch.Tensor = None,
) -> torch.Tensor
"""dropout_p should be set to 0.0 during evaluation
```

Supports multi-query and grouped-query attention (MQA/GQA) by passing in K, V with fewer heads than Q. Note that the number of heads in Q must be divisible by the number of heads in KV. For example, if Q has 6 heads and K, V have 2 heads, head 0, 1, 2 of Q will attention to head 0 of K, V, and head 3, 4, 5 of Q will attention to head 1 of K, V."

Pytorch 等价实现:

```
def ref_flash_attn_varlen_func(
    q: torch.Tensor,
    k: torch.Tensor,
    v: torch.Tensor,
    cu_seqlens_q: torch.Tensor,
    cu_seqlens_k: torch.Tensor,
    max_seqlen_q: int,
    max_seqlen_k: int,
    dropout_p: float = 0.0,
    softmax_scale: float = None,
    causal: bool = False,
    return_attn_probs: bool = False,
):
    if return_attn_probs:
        raise NotImplementedError("return_attn_probs not supported!")
    out = torch.zeros_like(q)
    def ref_masked_attention(query: torch.Tensor, key: torch.Tensor, value:
        torch.Tensor, scale: float, attn_mask=None,):
        query = query * scale
        attn = torch.einsum("qhd,khd->hqk", query, key)
        if attn_mask is not None:
            attn = attn + attn_mask
        attn = torch.softmax(attn, dim=-1)
        out = torch.einsum("hqk,khd->qhd", attn.to(query.dtype), value.to(query.dtype))
        return out
    def repeat_kv(x: torch.Tensor, n_rep: int) -> torch.Tensor:
        """torch.repeat_interleave(x, dim=2, repeats=n_rep)"""
        if len(x.shape) == 4:
            batch, seq_len, n_kv_heads, head_dim = x.shape
        elif len(x.shape) == 3:
            tokens, n_kv_heads, head_dim = x.shape
        if n_rep == 1:
            return x
        if len(x.shape) == 4:
            return (
                x[:, :, :, None, :]
                .expand(batch, seq_len, n_kv_heads, n_rep, head_dim)
                .reshape(batch, seq_len, n_kv_heads * n_rep, head_dim)
            )
        elif len(x.shape) == 3:
            return (
```

```

        x[:, :, None, :]
        .expand(tokens, n_kv_heads, n_rep, head_dim)
        .reshape(tokens, n_kv_heads * n_rep, head_dim)
    )
def un-
    ↪ pad_causal_torch(q,k,v,output,cu_seqlens_q,cu_seqlens_k,max_seq_len_q,max_seq_len_kv,dtype,at
    ↪ =True,):
    head_num = q.size(1)
    head_num_kv = k.size(1)
    head_dim = q.size(2)

    assert head_num % head_num_kv == 0

    # tokens,head_num,head_dim
    if head_num != head_num_kv:
        # k = k.repeat(1, head_num//head_num_kv, 1)#[0,1,2,0,1,2,0,1,2,0,1,2]
        # v = v.repeat(1, head_num//head_num_kv, 1)

        k = repeat_kv(k, head_num // head_num_kv) # [0,0,0,0,1,1,1,1,2,2,2,2] GROUP
        v = repeat_kv(v, head_num // head_num_kv)

    batch_size = cu_seqlens_q.size(0) - 1

    for i in range(batch_size):
        q_start_index = cu_seqlens_q[i]
        q_end_index = cu_seqlens_q[i + 1]
        cur_q_len = q_end_index - q_start_index
        # 1*seq_len,head_num,head_dim
        cur_q = q[q_start_index:q_end_index]

        k_start_index = cu_seqlens_k[i]
        k_end_index = cu_seqlens_k[i + 1]
        cur_k_len = k_end_index - k_start_index

        cur_k = k[k_start_index:k_end_index]
        cur_v = v[k_start_index:k_end_index]

        # mask = torch.tril(torch.ones([cur_q_len, cur_k_len],
        ↪ dtype=torch.bool)).cuda()
        # mask = mask.unsqueeze(0).unsqueeze(0)
        if is_causal:
            # Create attention mask.
            attn_mask = torch.triu(
                torch.ones(cur_q_len, cur_k_len, dtype=dtype), diagonal=1
            )
            attn_mask = attn_mask * torch.finfo(dtype).min
            attn_mask = attn_mask.to(dtype=dtype, device="cuda")
        else:
            attn_mask = None

```

```

        ref_output = ref_masked_attention(
            cur_q,
            cur_k,
            cur_v,
            atten_scale,
            attn_mask=attn_mask,
        )
        output[q_start_index:q_end_index].copy_(ref_output)
    unpad_causal_torch(
        q,
        k,
        v,
        out,
        cu_seqlens_q,
        cu_seqlens_k,
        max_seqlen_q,
        max_seqlen_k,
        torch.float16,
        softmax_scale,
        causal,
    )
    return out

```

参数说明

- q: (total_q, nheads, headdim), where total_q = total number of query tokens in the batch.
- k: (total_k, nheads_k, headdim), where total_k = total number of key tokens in the batch.
- v: (total_k, nheads_k, headdim), where total_k = total number of key tokens in the batch.
- cu_seqlens_q: (batch_size + 1,), dtype torch.int32. The cumulative sequence lengths of the sequences in the batch, used to index into q.
- cu_seqlens_k: (batch_size + 1,), dtype torch.int32. The cumulative sequence lengths of the sequences in the batch, used to index into kv.
- max_seqlen_q: int. Maximum query sequence length in the batch.
- max_seqlen_k: int. Maximum key sequence length in the batch.
- dropout_p: float. Dropout probability.
- softmax_scale: float. The scaling of QK^T before applying softmax. Default to $1 / \sqrt{\text{headdim}}$.
- causal: bool. Whether to apply causal attention mask (e.g., for auto-regressive modeling).
- return_attn_probs: bool. Whether to return the attention probabilities. This option is for testing only. The returned probabilities are not guaranteed to be correct (they might not have the right scaling).

返回值

- out: (total, nheads, headdim).
- softmax_lse [optional, if return_attn_probs=True]: (batch_size, nheads, seqlen). The logsumexp of each row of the matrix $QK^T * \text{scaling}$ (e.g., log of the softmax normalization factor).

- `S_dmask` [optional, if `return_attn_probs=True`]: (batch_size, nheads, seqlen, seqlen). The output of softmax (possibly with different scaling). It also encodes the dropout pattern (negative means that location was dropped, nonnegative means it was kept).

1.8.2.41 flash_attn_func

```
ixformer.functions.flash_attn_func(
    q: torch.Tensor,
    k: torch.Tensor,
    v: torch.Tensor,
    dropout_p: float = 0.0,
    softmax_scale: float = None,
    causal: bool = False,
    return_attn_probs: bool = False,
) -> torch.Tensor
# Pytorch 等价实现:

def ref_flash_attn_func(
    q: torch.Tensor,
    k: torch.Tensor,
    v: torch.Tensor,
    dropout_p: float = 0.0,
    softmax_scale: float = None,
    causal: bool = False,
    return_attn_probs: bool = False,
):
    def repeat_kv(x: torch.Tensor, n_rep: int) -> torch.Tensor:
        """torch.repeat_interleave(x, dim=2, repeats=n_rep)"""
        if len(x.shape) == 4:
            batch, seq_len, n_kv_heads, head_dim = x.shape
        elif len(x.shape) == 3:
            tokens, n_kv_heads, head_dim = x.shape
        if n_rep == 1:
            return x
        if len(x.shape) == 4:
            return (
                x[:, :, :, None, :]
                .expand(batch, seq_len, n_kv_heads, n_rep, head_dim)
                .reshape(batch, seq_len, n_kv_heads * n_rep, head_dim)
            )
        elif len(x.shape) == 3:
            return (
                x[:, :, None, :]
                .expand(tokens, n_kv_heads, n_rep, head_dim)
                .reshape(tokens, n_kv_heads * n_rep, head_dim)
            )

    def mha(q, k, v, atten_scale, is_causal):
        q = q.permute(0, 2, 1, 3).contiguous() # batch num_head seq_len head_dim
        k = k.permute(0, 2, 1, 3).contiguous()
```

```

v = v.permute(0, 2, 1, 3).contiguous()

# 2. q*kt softmax
scores_qk = torch.matmul(q.float(), k.float().transpose(-2, -1)) * atten_scale
q_seq_len = q.size(2)
kv_seq_len = k.size(2)
if is_causal:
    # Create attention mask.
    attn_mask = torch.triu(
        torch.ones(q_seq_len, kv_seq_len, dtype=torch.int), diagonal=1
    )
    attn_mask = attn_mask.to(dtype=torch.int, device="cuda")
else:
    attn_mask = None
# softmax
# print(scores_qk.shape, attn_mask.shape)
if attn_mask is not None:
    # print(scores_qk.shape, attn_mask.shape)
    scores_qk = scores_qk + attn_mask * (-100000)
scores_qk = torch.nn.functional.softmax(scores_qk, dim=-1)
# 3. x = qk_scores * v
scores_v = torch.matmul(scores_qk, v.float())
scores_v = scores_v.half()
return scores_v.permute(0, 2, 1, 3).contiguous()
if return_attn_probs:
    raise NotImplementedError("return_attn_probs not supported!")
head_num = q.size(2)
head_num_kv = k.size(2)
if head_num != head_num_kv:
    k = repeat_kv(k, head_num // head_num_kv) # [0,0,0,0,1,1,1,1,2,2,2,2] GROUP
    v = repeat_kv(v, head_num // head_num_kv)
output_pt = mha(q, k, v, softmax_scale, causal)
return output_pt

```

参数说明

returned probabilities are not guaranteed to be correct (they might not have the right scaling). - q: (batch_size, seqlen, nheads, headdim) - k: (batch_size, seqlen, nheads_k, headdim) - v: (batch_size, seqlen, nheads_k, headdim) - dropout_p: float. Dropout probability. - softmax_scale: float. The scaling of QK^T before applying softmax. Default to 1 / sqrt(headdim). - causal: bool. Whether to apply causal attention mask (e.g., for auto-regressive modeling). - return_attn_probs: bool. Whether to return the attention probabilities. This option is for testing only.

1.8.3 ixformer.functions (低精度算子)

1.8.3.1 i8w8o32

```
def i8w8o32(input: torch.Tensor, weight: torch.Tensor)
```

参数说明

- input - [bs,ic] dtype: int8
- weights - [oc,ic] dtype: int8

返回值

- Tensor - [bs,oc] dtype:int32

等价实现

```
def ref_i8w8o32(input: torch.Tensor, weight: torch.Tensor):
    output = torch.nn.functional.linear(input.float(), weight.float()).int()
    return output
```

1.8.3.2 w8a16

```
def w8a16(
    inputs: "torch.Tensor",
    qweights: "torch.Tensor",
    scales: "torch.Tensor",
    group_size: int = -1,
    format: str = "TN",
)
```

参数说明

当 `inputs.size(0) <= config.IXFORMER_GEMV_THRESHOLD`, 即 `bs <= config.IXFORMER_GEMV_THRESHOLD` 时, 使用 `w8a16_gemv` 接口, 否则使用 `w8a16_gemm` 接口

- w8a16 gemv 接口
 - input : bf16|fp16 (bs, ic)
 - qweights : int8 TN: (oc, ic) NN:(ic, oc)
 - scales : bf16|fp16 (ic/group_size, oc)
 - TN 支持条件: $ic \% groupSize = 0, oc \% 2 = 0, bs \leq 4$
 - NN 支持条件: $ic \% groupSize = 0, oc \% 4 = 0, bs \leq 4$
- w8a16 gemm 接口
 1. group_size=ic
 - input : bf16|fp16 (bs, ic)

- qweights : int8 TN:(oc, ic) NN:(ic, oc)
- scales : bf16|fp16 (1, oc)
- TN 支持条件: $k\%4 == 0$
- NN 支持条件: $ic\%64 == 0$ $oc\%64 == 0$

2. group_size=64

- input : bf16|fp16 (bs, ic)
- qweights : int8 TN:(oc, ic)
- scales : bf16|fp16 (ic/64, oc)
- TN 支持条件: $oc\%2 == 0$ $ic\%64 == 0$
- NN 不支持

返回值

- Tensor - [bs,oc], dtype:[torch.float16, torch.bfloat16]

等价实现

```
def ref_w8a16(
    inputs: "torch.Tensor",
    qweights: "torch.Tensor",
    scales: "torch.Tensor",
    group_size: int = -1,
    format: str = "TN",
):
    if format == "TN":
        weights = dequant(qweights.transpose(0, 1), scales, group_size)
    elif format == "NN":
        weights = dequant(qweights, scales, group_size)
    return torch.nn.functional.linear(inputs, weights.to(inputs.dtype))

def dequant(qweight, scales, group_size):
    IC, OC = qweight.shape
    weight = qweight.t().reshape(OC, -1, group_size).to(
        torch.float32
    ) * scales.t().unsqueeze(-1)
    return weight.reshape(OC, IC)
```

1.8.3.3 wu8a16

```
def wu8a16(
    inputs: "torch.Tensor",
    qweights: "torch.Tensor",
    scales: "torch.Tensor",
    zeros: "torch.Tensor",
    group_size: int = -1,
    format: str = "TN",
    persistent: int = 0,
)
```

参数说明

wu8a16 非对称量化 gemm 接口

1. group_size=ic

- input : bf16|fp16 (bs, ic)
- qweights : int8 TN:(oc, ic)
- scales : bf16|fp16 (1, oc)
- zeros : bf16|fp16 (1, oc)
- TN 支持条件: $ic \% 64 == 0$
- NN 不支持

2. group_size=64

- input : bf16|fp16 (bs, ic)
- qweights : int8 TN:(oc, ic)
- scales : bf16|fp16 (ic/64, oc)
- zeros : bf16|fp16 (ic/64, oc)
- TN 支持条件: $oc \% 2 == 0 \ \&\& \ ic \% 64 == 0$
- NN 不支持

返回值

- Tensor - [bs,oc], dtype:[torch.float16, torch.bfloat16]

等价实现

```
def ref_wu8a16(
    inputs: "torch.Tensor",
    qweights: "torch.Tensor",
    scales: "torch.Tensor",
    zeros: "torch.Tensor",
    group_size: int = -1,
    format: str = "TN",
):
    assert format in ["TN"]
    assert len(qweights.shape) == 2
    assert len(scales.shape) == 2
    org_w_shape = qweights.shape
    scales = scales.transpose(0, 1).flatten().view(-1, 1)
    zeros = zeros.transpose(0, 1).flatten().view(-1, 1)

    if group_size != -1:
        qweights = qweights.reshape(-1, group_size)
    w = (qweights - zeros) * scales
    w = w.reshape(org_w_shape)
    output = torch.matmul(inputs, w.t())
    return output
```

1.8.3.4 w8a8

```
def w8a8(
    inputs: "torch.Tensor",
    weights: "torch.Tensor",
    i_scales: "torch.Tensor",
    w_salces: "torch.Tensor",
    output: "torch.Tensor",
    format: str = "TN",
    persistent: int = 0,
)
```

参数说明

- inputs - [n,k], dtype: int8
- weights - TN: [m,k], NN: [k,m], dtype: int8
- i_scales - [n], dtype: [torch.float]
- w_salces - [m], dtype: [torch.float]
- output - [n,m], dtype: [torch.float16, torch.bfloat16]
- format - TN or NN
- persistent - 默认 0, 使用 overlap 需要设置为 1

限制条件: TN - m, n 需要有一个大于 64, $m \% 2 == 0$ $k \% 64 == 0$ $k \geq 256$ NN - $m \% 64 == 0$ $k \% 64 == 0$ $k \geq 256$

返回值

- output - [n,m], dtype:[torch.float16, torch.bfloat16]

等价实现

```
def ref_w8a8(
    inputs: "torch.Tensor",
    weights: "torch.Tensor",
    i_scales: "torch.Tensor",
    w_salces: "torch.Tensor",
    output: "torch.Tensor",
    format: str = "TN",
    persistent=0,
):
    dtype = output.dtype
    dq_input = inputs.to(dtype) * i_scales.view(-1, 1).to(dtype)
    if format == "TN":
        dq_weight = weights.to(dtype) * w_salces.view(-1, 1).to(dtype)
    else:
        dq_weight = weights.to(dtype) * w_salces.view(1, -1).to(dtype)
    assert format in ["TN", "NN"]
    if format == "TN":
        dq_weight = dq_weight.transpose(0, 1)
    return torch.matmul(dq_input, dq_weight, out=output)
```

1.8.3.5 wi4a16

```
def wi4a16(
    inputs: "torch.Tensor",
    qweights: "torch.Tensor",
    scales: "torch.Tensor",
    zeros: "torch.Tensor",
    group_size: int = -1,
    format: str = "TN",
    output=None,
)
```

参数说明

当 inputs.size(0)=1, 即 bs=1 时, 使用 wi4a16_gemv 接口, 否则使用 wi4a16_gemm 接口

- wi4a16 gemv 接口

支持条件:

- format = TN
- group_size = 128
- input : bf16|fp16 (bs, ic)
- qweights : int8 (oc/2, ic)
- scales : bf16|fp16 (ic/group_size, oc)
- zeros : bf16|fp16 (ic/group_size, oc)
- TN 支持条件: oc%2==0 && ic%128==0
- NN 不支持

- wi4a16 gemm 接口

支持条件:

- format = TN
- group_size = 128
- input : fp16 (bs, ic)
- qweights : int8 (oc/2, ic)
- scales : fp16 (ic/group_size, oc)
- zeros : fp16 (ic/group_size, oc)
- TN 支持条件: oc%2==0 && ic%128==0
- NN 不支持

返回值

- Tensor - [bs,oc], dtype:[torch.float16, torch.bfloat16]

等价实现

```
def ref_wi4a16(
    inputs: "torch.Tensor",
    qweights: "torch.Tensor",
```

```
scales: "torch.Tensor",
zeros: "torch.Tensor",
group_size: int = -1,
format: str = "TN",
):
    assert format in ["TN"]
    weights = dequant_weight(
        qweights,
        scales.transpose(0, 1).contiguous(),
        zeros.transpose(0, 1).contiguous(),
        group_size,
    )
    output = torch.nn.functional.linear(inputs, weights.to(inputs.dtype))
    return output
def dequant_weight(tensor, scales, zeros, block_size):
    # from CPM
    """
    tensor: (oc/2, ic)
    scales: (oc, ic/group_size)
    zeros: (oc, ic/group_size)
    """
    dtype = scales.dtype
    left = tensor >> 4
    right = tensor << 4 >> 4
    left, right = right, left
    ret = torch.cat((left, right), dim=-1).reshape(-1, left.size(-1))
    ret_shape = ret.size()
    ret = ret.view(-1, block_size)
    ret = scales.view(-1, 1) * (ret - zeros.view(-1, 1))
    ret = ret.reshape(ret_shape).to(dtype=dtype)
    return ret
```

1.9 附录 3: ixFormer 训练 Python API

ixFormer 训练 Python API 包含:

- `ixformer.train.functions` (基础算子)
- `ixformer.train.functions` (融合算子)

1.9.1 `ixformer.train.functions` (基础算子)

`layer_norm`, `matmul`, `linear`, `gelu` 基础算子在接口设计中保持同 PyTorch 一致的接口, 使用详情请参考 PyTorch 官方文档。

1.9.2 ixformer.train.functions (融合算子)

1.9.2.1 fused_apply_rotary_pos_emb

```
ixformer.train.functions.fused_apply_rotary_pos_emb(
    t: torch.Tensor,
    freqs: torch.Tensor,
    transpose_output_memory: bool = False,
) -> torch.Tensor

# Pytorch 等价实现:

def ref_fused_apply_rotary_pos_emb(
    t: torch.Tensor, freqs: torch.Tensor, transpose_output_memory: bool = False
):
    def _rotate_half(x: torch.Tensor) -> torch.Tensor:
        """Change sign so the last dimension becomes [-odd, +even]

        Args:
            x (Tensor): Input tensor

        Returns:
            Tensor: Tensor rotated half
        """
        x1, x2 = torch.chunk(x, 2, dim=-1)
        return torch.cat((-x2, x1), dim=-1)

    def apply_rotary_pos_emb(t: torch.Tensor, freqs: torch.Tensor) -> torch.Tensor:
        """Apply rotary positional embedding to input tensor T.

        check https://kexue.fm/archives/8265 for detailed formulas

        Args:
            t (Tensor): Input tensor T is of shape [seq_length, ..., dim]
            freqs (Tensor): Rotary Positional embedding tensor freq is of shape
                ↪ [seq_length, ..., dim]

        Returns:
            Tensor: The input tensor after applying RoPE
        """
        rot_dim = freqs.shape[-1]

        # ideally t_pass is empty so rotary pos embedding is applied to all tensor t
        t, t_pass = t[..., :rot_dim], t[..., rot_dim:]

        # first part is cosine component
        # second part is sine component, need to change signs with _rotate_half method
        cos_ = torch.cos(freqs).to(t.dtype)
        sin_ = torch.sin(freqs).to(t.dtype)
```

```
t = (t * cos_) + (_rotate_half(t) * sin_)
return torch.cat((t, t_pass), dim=-1)

output_unfused = apply_rotary_pos_emb(t, freqs)
return output_unfused
```

参数说明

Shape:

- t: [s,b,h,d]
- freqs: [s, 1, 1, d]
- transpose_output_memory: default to False. Whether to transpose the 's' and 'b'dimension of the output's underlying memory format. This is very helpful when you want to get a contiguous tensor after calling output.transpose(0, 1).

支持数据类型: float16, bfloat16, float32

1.9.2.2 geglu

```
ixformer.train.functions.geglu(input: torch.Tensor) -> torch.Tensor
```

Pytorch 等价实现:

```
import torch.nn.functional as NNF
def ref_gelu_and_mul(x: torch.Tensor) -> torch.Tensor:
    dtype = x.dtype
    x = x.float()
    x1, x2 = x.chunk(chunks=2, dim=-1)
    res = NNF.gelu(x2) * x1
    return res.to(dtype)
```

参数说明

Shape:

- input: [*,2c]
- output: [*,c]

支持数据类型: bfloat16, float16, float32

1.9.2.3 residual_bias

```
ixformer.train.functions.residual_bias(
    input: torch.Tensor,
    residual: torch.Tensor,
    bias: torch.Tensor = None,
```

```

    output: torch.Tensor = None,
    alpha: float = 1,
)-> torch.Tensor

# Pytorch 等价实现:
def ref_residual_bias(
    input: torch.Tensor,
    residual: torch.Tensor,
    bias: torch.Tensor = None,
    alpha: float = 1,
):
    if bias is not None:
        output = residual.float() * alpha + input.float() + bias.float()
    else:
        output = residual.float() * alpha + input.float()

    return output.to(residual.dtype)

```

参数说明

Shape:

- input: [batch_count, seq_len, hidden_size]
- residual: [batch_count, seq_len, hidden_size]
- bias: [hidden_size]
- output: [batch_count, seq_len, hidden_size]
- alpha: float, multiplier for residual

支持数据类型: float16

1.9.2.4 residual_bias_ln

```

ixformer.train.functions.residual_bias_ln(
    input: torch.Tensor,
    residual: torch.Tensor,
    bias: torch.Tensor,
    ln_weight: torch.Tensor,
    ln_bias: torch.Tensor,
    alpha=1,
    is_post_ln=True,
    output: torch.Tensor = None,
    training: bool = False,
)-> torch.Tensor

```

```

# Pytorch 等价实现:
def ref_residual_layer_norm(
    input: torch.Tensor,
    residual: torch.Tensor,

```

```

bias: torch.Tensor,
ln_weight: torch.Tensor,
ln_bias: torch.Tensor,
alpha=1,
is_post_ln=True,
output: torch.Tensor = None,
training: bool = False,
):
    dtype = input.dtype
    if bias is None:
        x = input.float() + residual.float() * alpha
    else:
        x = input.float() + residual.float() * alpha + bias.float()

    y = torch.nn.functional.layer_norm(
        x.to(dtype), [input.shape[-1]], ln_weight, ln_bias, eps=1e-5
    )

    if is_post_ln:
        return y, y
    else:
        return y, x.to(dtype)

```

参数说明

Shape:

- input: [batch_count * seq_len, hidden_size]
- residual: [batch_count * seq_len, hidden_size]
- bias: [hidden_size]
- ln_weight: [hidden_size]
- ln_bias: [hidden_size]
- alpha: float = 1.0
- output: [batch_count * seq_len, hidden_size]
- is_post_ln: bool, post-layernorm or pre-layernorm
- training: bool

支持数据类型: float16

1.9.2.5 swiglu

ixformer.train.functions.swiglu(input: torch.Tensor) -> torch.Tensor

Pytorch 等价实现:

```

def ref_silu_and_mul(x: torch.Tensor) -> torch.Tensor:
    dtype = x.dtype
    x = x.float()
    x1, x2 = x.chunk(chunks=2, dim=-1)

```

```
res = torch.nn.functional.silu(x1) * x2
return res.to(dtype)
```

参数说明

Shape:

- input: [*,2c]
- output: [*,c]

支持数据类型: bfloat16, float16, float32

1.9.2.6 vocab_parallel_cross_entropy

```
ixformer.train.functions.vocab_parallel_cross_entropy(
    vocab_parallel_logits: torch.Tensor,
    target: torch.Tensor,
    label_smoothing: float = 0.0,
    world_size: int = 1,
    vocab_start_index: int = 0,
    vocab_end_index: int = 320000,
    group=None,
)-> torch.Tensor
```

Pytorch 等价实现:

```
def ref_vocab_parallel_cross_entropy(
    vocab_parallel_logits: torch.Tensor,
    target: torch.Tensor,
    label_smoothing: float = 0.0,
    world_size: int = 1,
    vocab_start_index: int = 0,
    vocab_end_index: int = 320000,
    group=None,
):
    if world_size == 1:
        vocab_parallel_logits = vocab_parallel_logits.float()
        partition_vocab_size = vocab_parallel_logits.size()[-1]
        logits_max = torch.max(vocab_parallel_logits, dim=-1)[0]
        vocab_parallel_logits = vocab_parallel_logits - logits_max.unsqueeze(dim=-1)
        target_mask = (target < vocab_start_index) | (target >= vocab_end_index)
        masked_target = target.clone() - vocab_start_index
        masked_target[target_mask] = 0
        logits_2d = vocab_parallel_logits.view(-1, partition_vocab_size)
        masked_target_1d = masked_target.view(-1)
        arange_1d = torch.arange(
            start=0, end=logits_2d.size()[0], device=logits_2d.device
        )
        predicted_logits_1d = logits_2d[arange_1d, masked_target_1d]
```

```

predicted_logits_1d = predicted_logits_1d.clone().contiguous()

predicted_logits = predicted_logits_1d.view_as(target)
predicted_logits[target_mask] = 0.0
exp_logits = vocab_parallel_logits
torch.exp(vocab_parallel_logits, out=exp_logits)
sum_exp_logits = exp_logits.sum(dim=-1)
loss = torch.log(sum_exp_logits) - predicted_logits
exp_logits.div_(sum_exp_logits.unsqueeze(dim=-1))
if label_smoothing > 0:
    """
    We'd like to assign 1 / (K - 1) probability mass to every index that is not the
    ↪ ground truth.
    = (1 - alpha) * y_gt + alpha * mean(y_{i for i != gt})
    = (1 - alpha) * y_gt + (alpha / (K - 1)) * \sum_{i != gt} y_i
    = ((K - 1) * (1 - alpha) / (K - 1)) * y_gt + (alpha / (K - 1)) * \sum_{i != gt}
    ↪ y_i
    = (K * (1 - alpha) - 1) / (K - 1) * y_gt + (alpha / (K - 1)) * \sum_{i} y_i
    = (1 - (alpha * K) / (K - 1)) * y_gt + ((alpha * K) / (K - 1)) * \sum_{i} y_i
    ↪ / K

    From: https://github.com/NVIDIA/NeMo/blob/main/nemo/collections/common/losses/
    ↪ smoothed_cross_entropy.py
    """
    assert 1.0 > label_smoothing > 0.0
    smoothing = label_smoothing * vocab_size / (vocab_size - 1)

    # Exp logits at this point are normalized probabilities. So we can just take
    ↪ the log to get log-probs.
    log_probs = torch.log(exp_logits)
    mean_log_probs = log_probs.mean(dim=-1)
    loss = (1.0 - smoothing) * loss - smoothing * mean_log_probs
else:
    # Maximum value along vocab dimension across all GPUs.
    logits_max = torch.max(vocab_parallel_logits, dim=-1)[0]
    torch.distributed.all_reduce(
        logits_max, op=torch.distributed.ReduceOp.MAX, group=group
    )
    # Subtract the maximum value.
    vocab_parallel_logits = vocab_parallel_logits - logits_max.unsqueeze(dim=-1)

    # Get the partition's vocab indecies
    partition_vocab_size = vocab_parallel_logits.size()[-1]

    # Create a mask of valid vocab ids (1 means it needs to be masked).
    target_mask = (target < vocab_start_index) | (target >= vocab_end_index)
    masked_target = target.clone() - vocab_start_index
    masked_target[target_mask] = 0

    # Get predicted-logits = logits[target].

```

```

# For Simplicity, we convert logits to a 2-D tensor with size
# [*, partition-vocab-size] and target to a 1-D tensor of size [*].
logits_2d = vocab_parallel_logits.view(-1, partition_vocab_size)
masked_target_1d = masked_target.view(-1)
arange_1d = torch.arange(
    start=0, end=logits_2d.size()[0], device=logits_2d.device
)
predicted_logits_1d = logits_2d[arange_1d, masked_target_1d]
predicted_logits_1d = predicted_logits_1d.clone().contiguous()
predicted_logits = predicted_logits_1d.view_as(target)
predicted_logits[target_mask] = 0.0
# All reduce is needed to get the chunks from other GPUs.
torch.distributed.all_reduce(
    predicted_logits,
    op=torch.distributed.ReduceOp.SUM,
    group=group,
)

# Sum of exponential of logits along vocab dimension across all GPUs.
exp_logits = vocab_parallel_logits
torch.exp(vocab_parallel_logits, out=exp_logits)
sum_exp_logits = exp_logits.sum(dim=-1)
torch.distributed.all_reduce(
    sum_exp_logits,
    op=torch.distributed.ReduceOp.SUM,
    group=group,
)

# Loss = log(sum(exp(logits))) - predicted-logit.
loss = torch.log(sum_exp_logits) - predicted_logits

# Normalize and optionally smooth logits
exp_logits.div_(sum_exp_logits.unsqueeze(dim=-1))
vocab_size = exp_logits.size(-1)
if label_smoothing > 0:
    """
    We'd like to assign 1 / (K - 1) probability mass to every index that is not the
    ↪ ground truth.
    = (1 - alpha) * y_gt + alpha * mean(y_{i for i != gt})
    = (1 - alpha) * y_gt + (alpha / (K - 1)) * \sum_{i != gt} y_i
    = ((K - 1) * (1 - alpha) / (K - 1)) * y_gt + (alpha / (K - 1)) * \sum_{i != gt}
    ↪ y_i
    = (K * (1 - alpha) - 1) / (K - 1) * y_gt + (alpha / (K - 1)) * \sum_{i} y_i
    = (1 - (alpha * K) / (K - 1)) * y_gt + ( (alpha * K) / (K - 1) ) * \sum_{i} y_i
    ↪ / K
    From: https://github.com/NVIDIA/NeMo/blob/main/nemo/collections/common/losses/
    ↪ smoothed_cross_entropy.py
    """
    assert 1.0 > label_smoothing > 0.0

```

```

smoothing = label_smoothing * vocab_size / (vocab_size - 1)

# Exp logits at this point are normalized probabilities. So we can just take
# the log to get log-probs.
log_probs = torch.log(exp_logits)
mean_log_probs = log_probs.mean(dim=-1)
loss = (1.0 - smoothing) * loss - smoothing * mean_log_probs

return loss

```

参数说明

当 world_size = 1 时：目前只支持 batch_size = 1 的情况，当 batch_size > 1 时，计算不正确

Shape:

- vocab_parallel_logits: [seq_len, 1, vocab_size]
- target: [seq_len, 1], dtype: torch.int64
- label_smoothing: float, 默认为 0.0. 用于标签平滑
- vocab_start_index: int, vocab_size 的开始 index, 用于生成 target mask
- vocab_end_index: int, vocab_size 的结束 index, 用于生成 target mask
- world_size: 开启 TP 并行的数量
- group: TP 并行组

支持数据类型: bfloat16, float16, float32

1.9.2.7 fused_rms_norm_affine

```

ixformer.train.functions.fused_rms_norm_affine(
    input: torch.Tensor,
    weight: torch.Tensor,
    normalized_shape: List[int],
    eps=1e-6
    memory_efficient=False
) -> torch.Tensor

```

Pytorch 等价实现:

```

def rms_forward_torch(inputs, weight, eps):
    dtype = inputs.dtype
    shape = list(inputs.shape)
    inputs = inputs.float().view(-1, shape[-1])
    weight = weight.float().view(1, weight.shape[0])

    rsigma = torch.rsqrt(inputs.pow(2).mean(-1, keepdim=True) + eps)
    res = inputs * rsigma * weight
    res = res.view(*shape).to(dtype)

    return res, rsigma.view(-1)

```

参数说明

Shape:

- input: $[*,c]$
- weight: $[c]$
- normalized_shape: `List[int] = [c]`
- eps: a value added to the denominator for numerical stability. Default: $1e-6$
- output: $[*,c]$

支持数据类型: bfloat16, float16

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。